```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P04.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S04_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001ⓓ
    ///      Boot            UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P04.Boot
{
    /// &lt;summary&gt;
```

```
/// Boot     T001
/// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY_MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //              JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            if (!AdHubBoot.Wire()) return false;   // P04 AdHub slot table registration (O-20261010-1325 five-slot rollo
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P04.Boot
{
    /// &lt;summary&gt;
    ///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
    /// T001c  WX/TT     Channel   Editor      Boot
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
```

```
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK X006                  TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt; key  key      T001b cfg_i18n +  NUnit    &lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
```

```
        public void ShowBanner(string pointId) { }
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
namespace Sy.P04.Newbie
{
    /// &lt;summary&gt;
    /// S04    V1.4     ·O-20261010-1245
    ///            ·M5                  ——
    ///          /          /    /   /
    ///       U362  HUD         6
    /// &lt;/summary&gt;
    public sealed class MainUI : MonoBehaviour
    {
        static readonly Color PalRed = new Color(0.902f, 0.224f, 0.275f);
        static readonly Color PalBlue = new Color(0.271f, 0.482f, 0.616f);
        static readonly Color PalGold = new Color(1f, 0.843f, 0f);
        static readonly Color PalGreen = new Color(0.165f, 0.616f, 0.561f);
        static readonly Color PalBlack = new Color(0.114f, 0.114f, 0.114f);
        static readonly Color LockedGray = new Color(0.35f, 0.35f, 0.35f, 0.85f);
        Canvas _canvas;
        Sprite _square;
        Text _walletText;
        RectTransform _codexPanel;
        RectTransform _disclosurePanel;
        int _lastWallet = -1;
        public void StartHunt()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P04_scene_01", "P04_unit_01");
            _canvas = FindObjectOfType&lt;Canvas&gt;();
            _square = MakeSquareSprite();
            //
            var old = GameObject.Find("StartButton");
            if (old != null)
            {
                var rt = (RectTransform)old.transform;
                rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
```

```
            rt.anchoredPosition = new Vector2(0, 320);
            rt.sizeDelta = new Vector2(560, 130);
            var label = old.GetComponentInChildren&lt;Text&gt;();
            if (label != null)
            {
                label.text = "     ·     ";
                label.fontSize = 44;
            }
            var img = old.GetComponent&lt;Image&gt;();
            if (img != null) img.color = PalGreen;
        }
        BuildModeButtons();
        BuildWalletLine();
        BuildCodexPanel();
        BuildDisclosurePanel();
        SyncWallet(true);
    }
    void BuildModeButtons()
    {
        MakeModeButton("     ·    ", new Vector2(0, 160), PalRed, () =&gt;
        {
            //     =TD     T025b·X2366
            if (!TdTutorialData.TutorialDone)
            {
                Yaotide.YaotideBattleLaunch.StartTutorial();
                SceneManager.LoadScene("S04_Yaotide");
                return;
            }
            Yaotide.YaotideBattleLaunch.StartRandom();
            SceneManager.LoadScene("S04_Yaotide");
        });
        if (Yaotide.DailyRift.IsEligible())
        {
            MakeModeButton("     ·      ", new Vector2(0, 0), PalGold, () =&gt;
            {
                Yaotide.YaotideBattleLaunch.StartDailyRift();
                SceneManager.LoadScene("S04_Yaotide");
            });
        }
        MakeModeButton("    ", new Vector2(-300, -180), PalBlack, ToggleCodex);
        MakeModeButton("    ·   ", new Vector2(300, -180), PalBlack, () =&gt;
        {
            var wallet = Yaotide.SealCharmWallet.Load();
            if (wallet.CanExchange &amp;&amp; Yaotide.SealDisclosure.NeedsPopup())
            {
                _disclosurePanel.gameObject.SetActive(true);
                return;
            }
            DoExchange();
        });
```

```
}
void BuildWalletLine()
{
    _walletText = MakeText(_canvas.transform, "  0", 44, new Vector2(0, -320), TextAnchor.MiddleCenter, 500, 60)
    _walletText.color = PalGold;
    MakeModeButton("    ·   ", new Vector2(0, -400), PalGreen, () =&gt;
    {
        long now = System.DateTimeOffset.Now.ToUnixTimeMilliseconds();
        var claim = Yaotide.RiftOfflinePay.OfflineClaim.Open(now);
        if (!claim.Claimable)
        {
            ShowToast("     1   ");
            return;
        }
        int got = claim.TryCollect(false);
        if (got &gt; 0)
        {
            SyncWallet(true);
            ShowToast("    +" + got);
        }
    });
}
void SyncWallet(bool force)
{
    int balance = Yaotide.SealCharmWallet.Load().Balance;
    if (!force &amp;&amp; balance == _lastWallet) return;
    _lastWallet = balance;
    _walletText.text = "  " + balance + (balance &gt;= Yaotide.SealCharmWallet.ExchangeCost ? "     " : "");
}
void DoExchange()
{
    var wallet = Yaotide.SealCharmWallet.Load();
    int got = wallet.TryExchangeFair();
    SyncWallet(true);
    ShowToast(got &gt; 0
        ? "     " + (Yaotide.YaoCodex.ById(got)?.Name ?? "?") + "   "
        : wallet.CanExchange ? "    " : "    60    ");
}
/// &lt;summary&gt;         01  V1.3        ·               &lt;/summary&gt;
void BuildDisclosurePanel()
{
    _disclosurePanel = MakeOverlayPanel("DisclosurePanel");
    MakeText(_disclosurePanel, "      ", 64, new Vector2(0, 700), TextAnchor.MiddleCenter, 900, 84).color = PalG
    var body = MakeText(_disclosurePanel, Yaotide.SealDisclosure.PopupText(), 38, new Vector2(0, 160), TextAncho
    body.color = new Color(0.91f, 0.89f, 0.85f);
    MakeModeButtonCore("    ·     ", new Vector2(0, -560), PalGreen, () =&gt;
    {
        Yaotide.SealDisclosure.MarkSeen();
        _disclosurePanel.gameObject.SetActive(false);
        DoExchange();
```

```
        }, _disclosurePanel);
        MakeModeButtonCore("   ", new Vector2(0, -710), PalBlack, () =&gt;
        {
            Yaotide.SealDisclosure.MarkSeen();
            _disclosurePanel.gameObject.SetActive(false);
        }, _disclosurePanel);
        _disclosurePanel.gameObject.SetActive(false);
    }
    void BuildCodexPanel()
    {
        _codexPanel = MakeOverlayPanel();
        MakeText(_codexPanel, "     ·     ", 72, new Vector2(0, 760), TextAnchor.MiddleCenter, 900, 90).color = PalG
        var yaos = Yaotide.YaoCodex.All;
        for (int i = 0; i &lt; yaos.Count; i++)
        {
            int row = i / 2, col = i % 2;
            var y = yaos[i];
            bool sealedUp = Yaotide.YaoCodex.IsSealed(y.Id);
            var card = new GameObject("Codex_" + y.Id, typeof(RectTransform), typeof(Image));
            card.transform.SetParent(_codexPanel, false);
            var rt = (RectTransform)card.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(-300 + col * 600, 520 - row * 230);
            rt.sizeDelta = new Vector2(560, 200);
            var img = card.GetComponent&lt;Image&gt;();
            img.sprite = _square;
            img.color = sealedUp ? (y.Tier == Yaotide.YaoCodex.Tier.Gold ? PalGold
                : y.Tier == Yaotide.YaoCodex.Tier.Purple ? PalRed : PalGreen) : LockedGray;
            MakeText(card.transform, sealedUp ? y.Name : "    " + y.SealHint + " ", 48,
                new Vector2(0, 50), TextAnchor.MiddleCenter, 520, 70).color = sealedUp ? PalBlack : new Color(0.7f,
            MakeText(card.transform, sealedUp ? y.Role : "   ", 38,
                new Vector2(0, -50), TextAnchor.MiddleCenter, 520, 60).color = sealedUp
                    ? new Color(0.1f, 0.1f, 0.1f, 0.75f) : new Color(0.55f, 0.55f, 0.5f);
        }
        MakeModeButtonCore("    ", new Vector2(0, -450), PalBlack, () =&gt;
            _disclosurePanel.gameObject.SetActive(true), _codexPanel);
        var close = MakeModeButtonCore("  ", new Vector2(0, -740), PalGreen, ToggleCodex);
        close.transform.SetParent(_codexPanel, false);
        _codexPanel.gameObject.SetActive(false);
    }
    void ToggleCodex()
    {
        bool show = !_codexPanel.gameObject.activeSelf;
        if (show)
        {
            var yaos = Yaotide.YaoCodex.All;
            for (int i = 0; i &lt; yaos.Count; i++)
            {
                var card = _codexPanel.Find("Codex_" + yaos[i].Id);
                if (card == null) continue;
```

```
            var img = card.GetComponent&lt;Image&gt;();
            bool sealedUp = Yaotide.YaoCodex.IsSealed(yaos[i].Id);
            var y = yaos[i];
            img.color = sealedUp ? (y.Tier == Yaotide.YaoCodex.Tier.Gold ? PalGold
                : y.Tier == Yaotide.YaoCodex.Tier.Purple ? PalRed : PalGreen) : LockedGray;
        }
    }
    _codexPanel.gameObject.SetActive(show);
}
void ShowToast(string message)
{
    _walletText.text = message;
    CancelInvoke(nameof(RestoreWallet));
    Invoke(nameof(RestoreWallet), 1.6f);
}
void RestoreWallet() =&gt; SyncWallet(true);
// ——— UI        UGUI·    YAML     ———
Button MakeModeButton(string text, Vector2 pos, Color color, System.Action onClick)
    =&gt; MakeModeButtonCore(text, pos, color, onClick, _canvas.transform);
Button MakeModeButtonCore(string text, Vector2 pos, Color color, System.Action onClick, Transform parent = null)
{
    var go = new GameObject("ModeBtn_" + text, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent != null ? parent : _canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(560, 130);
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = color;
    var label = MakeText(go.transform, text, 44, Vector2.zero, TextAnchor.MiddleCenter, 560, 130);
    label.color = color == PalGold ? PalBlack : new Color(0.96f, 0.94f, 0.9f);
    var btn = go.GetComponent&lt;Button&gt;();
    btn.onClick.AddListener(() =&gt; onClick());
    return btn;
}
RectTransform MakeOverlayPanel(string goName = "CodexPanel")
{
    var go = new GameObject(goName, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = new Color(0.05f, 0.03f, 0.08f, 0.96f);
    return rt;
}
static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
```

```
        {
            var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(w, h);
            var t = go.GetComponent&lt;Text&gt;();
            t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            t.text = text;
            t.fontSize = size;
            t.alignment = anchor;
            t.color = new Color(0.91f, 0.89f, 0.85f);
            t.supportRichText = true;
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// AdHubBoot.cs
using Sy.AdHub;
namespace Sy.P04.Boot
{
    /// &lt;summary&gt;
    /// P04 slot table for the shared AdHub layer (O-20261010-1325 rollout, design case five slots).
    /// Tuning numbers live in the JSON table below (data, not logic). Battle-run frequency band
    /// per design case: daily cap 3-5 plays, cooldown 60-90s. Every reward face stays &lt;= 2x gain.
    /// &lt;/summary&gt;
    public static class AdHubBoot
    {
        // Design case slots: seal_topup / rift_extra / offline_x2 / elite_lure / settle_x2.
        const string SlotTableJson =
            "{\"slots\":[" +
            "{\"slot\":\"seal_topup\",\"rewardFace\":\"same-day seal charm top-up\",\"dailyCap\":3,\"cooldownSec\":75,\"
            "{\"slot\":\"rift_extra\",\"rewardFace\":\"one extra daily rift run\",\"dailyCap\":4,\"cooldownSec\":75,\"in
            "{\"slot\":\"offline_x2\",\"rewardFace\":\"offline charm income x2\",\"dailyCap\":3,\"cooldownSec\":75,\"int
            "{\"slot\":\"elite_lure\",\"rewardFace\":\"next run elite yao encounter boost\",\"dailyCap\":5,\"cooldownSec
            "{\"slot\":\"settle_x2\",\"rewardFace\":\"run settlement income x2\",\"dailyCap\":4,\"cooldownSec\":75,\"int
        public const int SlotCount = 5;
        public static AdSlotTable Table =&gt; AdSlotTable.FromJson(SlotTableJson);
        /// &lt;summary&gt;Boot step 6 hook: registers game id and slot table. False only on a broken table.&lt;/summary
        public static bool Wire()
```

```
        {
            var table = Table;
            if (table.slots == null || table.slots.Length != SlotCount) return false;
            // Fully qualified: inside namespace Sy.P04.*, the sibling namespace Sy.AdHub shadows
            // the using-imported class AdHub during simple-name lookup (CS0234).
            Sy.AdHub.AdHub.SetGameId("P04");
            Sy.AdHub.AdHub.SetSlotTable(table);
            return true;
        }
    }
}
// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P04.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S04_Main";
        public const string Battle = "S04_Battle";
    }
    /// &lt;summary&gt;
    /// S04_Battle       IdleSim+TutorialFlow+BattleView   Tick
    ///    Sy.Telemetry Editor =Debug sink T001e        +TCB
    /// HUD                     6
    /// &lt;/summary&gt;
    public sealed class BattleLauncher : MonoBehaviour
    {
        IdleSim _sim;
        TutorialFlow _tutorial;
        BattleView _view;
        Sy.Telemetry.TelemetryService _telemetry;
        int _lastFloor = -1, _lastCharms = -1, _lastKills = -1, _lastCodex = -1;
        void Awake()
        {
            Application.targetFrameRate = 30;
            var canvas = FindObjectOfType&lt;Canvas&gt;();
            _sim = new IdleSim();
            _view = gameObject.AddComponent&lt;BattleView&gt;();
            _view.Initialize(canvas, _sim);
            _telemetry = new Sy.Telemetry.TelemetryService();
            _telemetry.AddSink(new TelemetryDebugSink());
            _tutorial = new TutorialFlow(
                _sim,
                hint =&gt;
                {
                    Debug.Log("[HINT] " + hint);
                    _view.SetHint(hint);
                },
                step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
```

```
            new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
    _view.SealRequested += OnSealRequested;
    _view.FormationPlaced += OnFormationPlaced;
    _sim.StateChanged += OnStateChanged;
}
void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Floor != _lastFloor) { _lastFloor = _sim.Floor; _view.SetFloor(_lastFloor); }
    if (_sim.SealCharms != _lastCharms) { _lastCharms = _sim.SealCharms; _view.SetCharms(_lastCharms); }
    if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
    if (_sim.CodexLitCount != _lastCodex) { _lastCodex = _sim.CodexLitCount; _view.SetCodex(_lastCodex); }
}
void OnSealRequested()
{
    _sim.PressSeal();
}
void OnFormationPlaced()
{
    _sim.PlaceGudiao();
}
void OnStateChanged(IdleSim.IdleState state)
{
    switch (state)
    {
        case IdleSim.IdleState.Fighting:
            _view.HideIntro();
            break;
        case IdleSim.IdleState.SealAnim:
            _view.PlaySealAnim();
            break;
        case IdleSim.IdleState.SealFailAnim:
            _view.PlaySealFail();
            break;
        case IdleSim.IdleState.FloorSkip:
            _view.ShowFloorSkip(true);
            break;
        case IdleSim.IdleState.WallDemo:
            _view.ShowFloorSkip(false);
            _view.ShowWallBanner(true);
            break;
        case IdleSim.IdleState.Formation:
            _view.ShowWallBanner(false);
            _view.ShowFormationPanel(true);
            break;
        case IdleSim.IdleState.BossFight:
            _view.ShowFormationPanel(false);
            break;
        case IdleSim.IdleState.Settled:
```

```
                    _view.ShowSettle(_sim.Kills, _sim.SealCharms, _sim.CodexLitCount);
                    break;
            }
        }
        sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
        {
            public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
            {
                var sb = new System.Text.StringBuilder(evtId);
                foreach (var kv in fields)
                {
                    sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
                }
                Debug.Log("[TELEMETRY] " + sb);
            }
        }
    }
}
// BattleView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P04.Newbie
{
    /// &lt;summary&gt;
    ///                UGUI——        / / / /
    /// X025        =        0.8s+        § V1.2      =    0.5s+
    ///   =                    4-9           =
    ///   UI Initialize          YAML
    /// &lt;/summary&gt;
    public sealed class BattleView : MonoBehaviour
    {
        //         x=-350     x=+500           0.3
        const float FriendX = -350f;
        const float FieldSpan = 850f;
        static readonly float[] RowY = { 300f, 0f, -300f };
        Canvas _canvas;
        Sprite _square;
        RectTransform _field;
        readonly Image[] _friendHpFills = new Image[NewbieConfig.FriendCount];
        Text _floorText, _charmText, _killsText, _hintText;
        Button _sealButton;
        Image _sealButtonImage;
        Text _sealLabel;
        RectTransform _introOverlay, _sealOverlay, _failOverlay, _skipBanner, _wallBanner, _formationPanel, _settlePanel
        RectTransform _magicRing;      //      6
        Image _sealFlash;               //
        Text _settleStats;
```

```
readonly Image[] _codexSlots = new Image[NewbieConfig.CodexSlots];
readonly Text[] _codexLabels = new Text[NewbieConfig.CodexSlots];
readonly Dictionary&lt;int, EnemyVisual&gt; _enemies = new();
readonly List&lt;int&gt; _scratch = new();
public event Action SealRequested;
public event Action FormationPlaced;
static readonly Color PalRed = new Color(0.902f, 0.224f, 0.275f);   //     #E63946
static readonly Color PalBlue = new Color(0.271f, 0.482f, 0.616f);  //     #457B9D
static readonly Color PalGold = new Color(1f, 0.843f, 0f);           //     #FFD700
static readonly Color PalGreen = new Color(0.165f, 0.616f, 0.561f);  //     #2A9D8F
static readonly Color PalBlack = new Color(0.114f, 0.114f, 0.114f); //    #1D1D1D
static readonly Color PalFrostedGold = new Color(0.63f, 0.55f, 0.31f); //    ·         §
static readonly Color PanelBg = new Color(0.07f, 0.08f, 0.10f, 0.95f);
public void Initialize(Canvas canvas, IdleSim sim)
{
    _canvas = canvas;
    _square = MakeSquareSprite();
    BuildBackdrop();
    BuildHud();
    BuildField();
    BuildCodex();
    BuildIntroOverlay();
    BuildSealOverlay();
    BuildFailOverlay();
    BuildSkipBanner();
    BuildWallBanner();
    BuildFormationPanel();
    BuildSettlePanel();
    SetHint("     ·    ……");
}
// ————    ————
/// &lt;summary&gt;X027 generated art: fullscreen backdrop behind all UI; skipped
/// entirely when the art key is missing so headless checks stay green.&lt;/summary&gt;
void BuildBackdrop()
{
    var art = Sy.Core.ArtRuntime.Get("P04_scene_01");
    if (art == null) return;
    var go = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    rt.SetAsFirstSibling();
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = art;
    image.color = Color.white;
    image.preserveAspect = true;
    image.raycastTarget = false;
}
```

```
void BuildHud()
{
    var root = _canvas.transform;
    _floorText = MakeText(root, " 0 ", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 300, 70);
    _floorText.color = PalGold;
    MakeText(root, "   " + NewbieConfig.StartLife, 46, new Vector2(0, 880), TextAnchor.UpperRight, 340, 70);
    _charmText = MakeText(root, "    ×" + NewbieConfig.StartSealCharms, 44, new Vector2(0, 820), TextAnchor.Uppe
    _charmText.color = PalGold;
    _killsText = MakeText(root, "   0", 44, new Vector2(0, 820), TextAnchor.UpperRight, 400, 60);
    _hintText = MakeText(root, "", 40, new Vector2(0, -560), TextAnchor.MiddleCenter, 1020, 150);
    _hintText.color = new Color(0.95f, 0.87f, 0.6f);
    var btnGo = new GameObject("SealButton", typeof(Image), typeof(Button));
    btnGo.transform.SetParent(root, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0f);
    btnRt.anchoredPosition = new Vector2(0, 140);
    btnRt.sizeDelta = new Vector2(460, 140);
    _sealButtonImage = btnGo.GetComponent&lt;Image&gt;();
    _sealButtonImage.sprite = _square;
    _sealButtonImage.color = new Color(0.35f, 0.35f, 0.38f);
    _sealButton = btnGo.GetComponent&lt;Button&gt;();
    _sealButton.interactable = false;
    _sealLabel = MakeText(btnGo.transform, "    2   ", 46, Vector2.zero, TextAnchor.MiddleCenter, 460, 140);
    _sealLabel.color = new Color(0.7f, 0.7f, 0.7f);
    _sealButton.onClick.AddListener(() =&gt; SealRequested?.Invoke());
}
void BuildField()
{
    var fieldGo = new GameObject("Field", typeof(RectTransform), typeof(Image));
    fieldGo.transform.SetParent(_canvas.transform, false);
    _field = (RectTransform)fieldGo.transform;
    _field.anchorMin = _field.anchorMax = new Vector2(0.5f, 0.5f);
    _field.anchoredPosition = new Vector2(0, 60);
    _field.sizeDelta = new Vector2(1040, 1360);
    var fieldImg = fieldGo.GetComponent&lt;Image&gt;();
    fieldImg.sprite = _square;
    fieldImg.color = PanelBg;
    var lineGo = new GameObject("FriendLine", typeof(RectTransform), typeof(Image));
    lineGo.transform.SetParent(_field, false);
    var lineRt = (RectTransform)lineGo.transform;
    lineRt.anchorMin = lineRt.anchorMax = new Vector2(0.5f, 0.5f);
    lineRt.anchoredPosition = new Vector2(FriendX - 110f, 0);
    lineRt.sizeDelta = new Vector2(6, 1240);
    var lineImg = lineGo.GetComponent&lt;Image&gt;();
    lineImg.sprite = _square;
    lineImg.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0.5f);
    for (int i = 0; i &lt; NewbieConfig.FriendCount; i++)
    {
        var go = new GameObject("Friend_" + i, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_field, false);
```

```
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(FriendX, RowY[i]);
            rt.sizeDelta = new Vector2(150, 150);
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = _square;
            image.color = i == 0 ? PalGreen : i == 1 ? PalBlue : new Color(0.85f, 0.78f, 0.55f);
            MakeText(go.transform, NewbieConfig.FriendNames[i].Substring(0, 1), 52, new Vector2(0, -100), TextAnchor
            var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
            hpGo.transform.SetParent(go.transform, false);
            var hpRt = (RectTransform)hpGo.transform;
            hpRt.anchorMin = new Vector2(0f, 1f);
            hpRt.anchorMax = new Vector2(1f, 1f);
            hpRt.anchoredPosition = new Vector2(0, -6);
            hpRt.sizeDelta = new Vector2(-8, 10);
            _friendHpFills[i] = hpGo.GetComponent&lt;Image&gt;();
            _friendHpFills[i].sprite = _square;
            _friendHpFills[i].type = Image.Type.Filled;
            _friendHpFills[i].fillMethod = Image.FillMethod.Horizontal;
            _friendHpFills[i].color = PalGreen;
        }
    }
    void BuildCodex()
    {
        var root = _canvas.transform;
        MakeText(root, "   ", 36, new Vector2(400, 750), TextAnchor.UpperCenter, 200, 50).color = PalGold;
        for (int i = 0; i &lt; NewbieConfig.CodexSlots; i++)
        {
            var go = new GameObject("Codex_" + i, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(root, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(400, 620 - i * 170);
            rt.sizeDelta = new Vector2(140, 140);
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = _square;
            image.color = new Color(0.16f, 0.16f, 0.18f);
            _codexSlots[i] = image;
            _codexLabels[i] = MakeText(go.transform, "?", 60, Vector2.zero, TextAnchor.MiddleCenter, 140, 140);
            _codexLabels[i].color = new Color(0.45f, 0.45f, 0.48f);
        }
    }
    void BuildIntroOverlay()
    {
        _introOverlay = MakeOverlayPanel("IntroOverlay", new Color(0.03f, 0.03f, 0.05f, 0.92f));
        MakeText(_introOverlay, "          ", 90, new Vector2(0, 180), TextAnchor.MiddleCenter, 1000, 120).color = P
        MakeText(_introOverlay, "             ——       ", 52, new Vector2(0, 40), TextAnchor.MiddleCenter, 1040, 90)
    }
    void BuildSealOverlay()
    {
```

```
    _sealOverlay = MakeOverlayPanel("SealOverlay", new Color(0.05f, 0.04f, 0.02f, 0.85f));
    //      6     § V1.2    =  +  0.8s+
    var ringGo = new GameObject("MagicRing", typeof(RectTransform));
    ringGo.transform.SetParent(_sealOverlay, false);
    _magicRing = (RectTransform)ringGo.transform;
    _magicRing.anchorMin = _magicRing.anchorMax = new Vector2(0.5f, 0.5f);
    _magicRing.anchoredPosition = new Vector2(0, 120);
    _magicRing.sizeDelta = new Vector2(600, 600);
    Color[] ringColors = { PalRed, PalBlue, PalGold, PalGreen, PalRed, PalBlue };
    for (int i = 0; i &lt; 6; i++)
    {
        var go = new GameObject("Shard_" + i, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_magicRing, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        float ang = i * 60f * Mathf.Deg2Rad;
        rt.anchoredPosition = new Vector2(Mathf.Cos(ang), Mathf.Sin(ang)) * 240f;
        rt.sizeDelta = new Vector2(70, 110);
        rt.localRotation = Quaternion.Euler(0f, 0f, i * 60f + 90f);
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = ringColors[i];
    }
    var flashGo = new GameObject("Flash", typeof(RectTransform), typeof(Image));
    flashGo.transform.SetParent(_sealOverlay, false);
    var flashRt = (RectTransform)flashGo.transform;
    flashRt.anchorMin = flashRt.anchorMax = new Vector2(0.5f, 0.5f);
    flashRt.anchoredPosition = new Vector2(0, 120);
    flashRt.sizeDelta = new Vector2(900, 900);
    _sealFlash = flashGo.GetComponent&lt;Image&gt;();
    _sealFlash.sprite = _square;
    _sealFlash.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0f);
    MakeText(_sealOverlay, "     ·   ", 72, new Vector2(0, -280), TextAnchor.MiddleCenter, 1000, 100).color = Pa
    _sealOverlay.gameObject.SetActive(false);
}
void BuildFailOverlay()
{
    _failOverlay = MakeOverlayPanel("FailOverlay", new Color(0.08f, 0.08f, 0.09f, 0.8f));
    MakeText(_failOverlay, "    ", 84, new Vector2(0, 200), TextAnchor.MiddleCenter, 1000, 110).color = new Colo
    MakeText(_failOverlay, "    ——         ×1     50% \n   " + NewbieConfig.PityGuaranteeCount + "     ", 46, ne
    _failOverlay.gameObject.SetActive(false);
}
void BuildSkipBanner()
{
    var go = new GameObject("SkipBanner", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    _skipBanner = (RectTransform)go.transform;
    _skipBanner.anchorMin = new Vector2(0f, 1f);
    _skipBanner.anchorMax = new Vector2(1f, 1f);
    _skipBanner.pivot = new Vector2(0.5f, 1f);
```

```
        _skipBanner.anchoredPosition = Vector2.zero;
        _skipBanner.sizeDelta = new Vector2(0, 340);
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = new Color(0.10f, 0.38f, 0.35f, 0.9f);
        MakeText(_skipBanner, "      ", 80, new Vector2(0, 190), TextAnchor.MiddleCenter, 1000, 100);
        MakeText(_skipBanner, " " + NewbieConfig.FloorSkipFrom + "  →  " + NewbieConfig.FloorSkipTo + "
        _skipBanner.gameObject.SetActive(false);
    }
    void BuildWallBanner()
    {
        var go = new GameObject("WallBanner", typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        _wallBanner = (RectTransform)go.transform;
        _wallBanner.anchorMin = new Vector2(0f, 1f);
        _wallBanner.anchorMax = new Vector2(1f, 1f);
        _wallBanner.pivot = new Vector2(0.5f, 1f);
        _wallBanner.anchoredPosition = Vector2.zero;
        _wallBanner.sizeDelta = new Vector2(0, 420);
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = new Color(0.45f, 0.11f, 0.13f, 0.88f);
        MakeText(_wallBanner, " 10  ·  ", 80, new Vector2(0, 130), TextAnchor.MiddleCenter, 1000, 100);
        MakeText(_wallBanner, "   10% ——               ", 40, new Vector2(0, 20), TextAnchor.MiddleCenter, 1000, 70)
        var btnGo = new GameObject("HeavenAidButton", typeof(RectTransform), typeof(Image), typeof(Button));
        btnGo.transform.SetParent(_wallBanner, false);
        var btnRt = (RectTransform)btnGo.transform;
        btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
        btnRt.anchoredPosition = new Vector2(0, -110);
        btnRt.sizeDelta = new Vector2(380, 100);
        var btnImg = btnGo.GetComponent&lt;Image&gt;();
        btnImg.sprite = _square;
        btnImg.color = new Color(0.35f, 0.35f, 0.35f, 0.8f);   //
        var btn = btnGo.GetComponent&lt;Button&gt;();
        btn.interactable = false;
        MakeText(btnGo.transform, "       ", 40, Vector2.zero, TextAnchor.MiddleCenter, 380, 100).color = new Color(
        _wallBanner.gameObject.SetActive(false);
    }
    void BuildFormationPanel()
    {
        _formationPanel = MakeOverlayPanel("FormationPanel", new Color(0.04f, 0.05f, 0.07f, 0.94f));
        MakeText(_formationPanel, "    ·   ", 72, new Vector2(0, 620), TextAnchor.MiddleCenter, 1000, 100).color = P
        MakeText(_formationPanel, "        ·      ", 42, new Vector2(0, 460), TextAnchor.MiddleCenter, 1000, 70);
        for (int i = 0; i &lt; NewbieConfig.FriendCount; i++)
        {
            var slot = MakeSlot(_formationPanel, new Vector2(-300 + i * 300, 300), PalGreen);
            MakeText(slot, NewbieConfig.FriendNames[i].Substring(0, 1), 56, Vector2.zero, TextAnchor.MiddleCenter, 2
        }
        MakeText(_formationPanel, "   · 1   ", 42, new Vector2(0, 90), TextAnchor.MiddleCenter, 1000, 70).color = Pa
        var btnGo = new GameObject("PlaceGudiaoButton", typeof(RectTransform), typeof(Image), typeof(Button));
```

```
    btnGo.transform.SetParent(_formationPanel, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0.5f);
    btnRt.anchoredPosition = new Vector2(0, -80);
    btnRt.sizeDelta = new Vector2(300, 300);
    var btnImg = btnGo.GetComponent&lt;Image&gt;();
    btnImg.sprite = _square;
    btnImg.color = PalFrostedGold;   //    ·         §
    MakeText(btnGo.transform, " ", 80, new Vector2(0, 40), TextAnchor.MiddleCenter, 300, 200).color = PalBlack;
    MakeText(btnGo.transform, "       ", 36, new Vector2(0, -100), TextAnchor.MiddleCenter, 300, 80).color = Pal
    btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt;
    {
        _formationPanel.gameObject.SetActive(false);
        FormationPlaced?.Invoke();
    });
    MakeText(_formationPanel, "                    ——       +18%", 36, new Vector2(0, -320), TextAnchor.MiddleCen
    _formationPanel.gameObject.SetActive(false);
}
void BuildSettlePanel()
{
    _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.04f, 0.05f, 0.07f, 0.96f));
    MakeText(_settlePanel, "      ·      ", 76, new Vector2(0, 520), TextAnchor.MiddleCenter, 1040, 110).color =
    _settleStats = MakeText(_settlePanel, "", 44, new Vector2(0, 280), TextAnchor.MiddleCenter, 1020, 340);
    MakeActionButton(_settlePanel, "    ", new Vector2(-260, -300), () =&gt; UnityEngine.SceneManagement.SceneMa
    MakeActionButton(_settlePanel, "     ", new Vector2(260, -300), () =&gt; UnityEngine.SceneManagement.SceneMa
    _settlePanel.gameObject.SetActive(false);
}
// ——————               reconcile           ——————
public void SyncView(IdleSim sim)
{
    ReconcileEnemies(sim);
    foreach (var kv in _enemies)
    {
        var v = kv.Value;
        v.Rt.anchoredPosition = new Vector2(FriendX + v.Pos * FieldSpan, RowY[v.Row]);
        v.Fill.fillAmount = v.MaxHp &gt; 0f ? v.Hp / v.MaxHp : 0f;
        if (v.SealReady != v.LastSealReady)
        {
            v.LastSealReady = v.SealReady;
            v.SealLabel.gameObject.SetActive(v.SealReady);
            v.Fill.color = v.SealReady ? PalGold : new Color(0.2f, 0.9f, 0.3f);
            v.LastAttempts = -1;
        }
        if (v.SealReady &amp;&amp; v.LastAttempts != v.Attempts)
        {
            v.LastAttempts = v.Attempts;
            int next = v.Attempts + 1;
            v.SealLabel.text = next &gt;= NewbieConfig.PityGuaranteeCount
                ? "     " + next + "/" + NewbieConfig.PityGuaranteeCount + " ·   "
                : "     " + next + "/" + NewbieConfig.PityGuaranteeCount + "  ";
```

```
            }
            if (v.Shielded != v.LastShielded)
            {
                v.LastShielded = v.Shielded;
                v.ShieldLabel.gameObject.SetActive(v.Shielded);
            }
        }
        for (int i = 0; i &lt; _friendHpFills.Length; i++)
        {
            var f = sim.Friends[i];
            _friendHpFills[i].fillAmount = f.MaxHp &gt; 0f ? f.Hp / f.MaxHp : 0f;
        }
        bool canSeal = sim.SealReadyEnemy != null;
        if (_sealButton.interactable != canSeal)
        {
            _sealButton.interactable = canSeal;
            _sealButtonImage.color = canSeal ? PalGold : new Color(0.35f, 0.35f, 0.38f);
            _sealLabel.color = canSeal ? PalBlack : new Color(0.7f, 0.7f, 0.7f);
        }
    }
    void ReconcileEnemies(IdleSim sim)
    {
        _scratch.Clear();
        foreach (var kv in _enemies)
            if (!ContainsEnemy(sim, kv.Key)) _scratch.Add(kv.Key);
        for (int i = 0; i &lt; _scratch.Count; i++)
        {
            Destroy(_enemies[_scratch[i]].Go);
            _enemies.Remove(_scratch[i]);
        }
        for (int i = 0; i &lt; sim.Enemies.Count; i++)
        {
            var e = sim.Enemies[i];
            if (!_enemies.TryGetValue(e.Id, out var v))
            {
                v = CreateEnemyVisual(e);
                _enemies[e.Id] = v;
            }
            v.Pos = e.Pos;
            v.Row = e.Row;
            v.Hp = e.Hp;
            v.MaxHp = e.MaxHp;
            v.SealReady = e.SealReady;
            v.Shielded = e.Shielded;
            v.Attempts = e.SealAttempts;
        }
    }
    static bool ContainsEnemy(IdleSim sim, int id)
    {
        for (int i = 0; i &lt; sim.Enemies.Count; i++)
```

```
            if (sim.Enemies[i].Id == id) return true;
        return false;
    }
    EnemyVisual CreateEnemyVisual(IdleSim.EnemyInst e)
    {
        var go = new GameObject("Enemy_" + e.Id, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_field, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.sizeDelta = e.Boss ? new Vector2(200, 200) : e.Collectible ? new Vector2(160, 160) : new Vector2(90, 90);
        var image = go.GetComponent&lt;Image&gt;();
        var bossArt = e.Boss ? Sy.Core.ArtRuntime.Get("P04_unit_01") : null;
        if (bossArt != null)
        {
            image.sprite = bossArt;
            image.color = Color.white;
            image.preserveAspect = true;
        }
        else
        {
            image.sprite = _square;
            image.color = e.Boss ? PalBlue : e.Collectible ? PalGold : PalRed;   //       ·
        }
        if (e.Collectible)
            MakeText(go.transform, e.Name.Substring(0, 1), 64, Vector2.zero, TextAnchor.MiddleCenter, 200, 200).colo
        var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
        hpGo.transform.SetParent(go.transform, false);
        var hpRt = (RectTransform)hpGo.transform;
        hpRt.anchorMin = new Vector2(0f, 1f);
        hpRt.anchorMax = new Vector2(1f, 1f);
        hpRt.anchoredPosition = new Vector2(0, -8);
        hpRt.sizeDelta = new Vector2(-10, 12);
        var hp = hpGo.GetComponent&lt;Image&gt;();
        hp.sprite = _square;
        hp.type = Image.Type.Filled;
        hp.fillMethod = Image.FillMethod.Horizontal;
        hp.color = new Color(0.2f, 0.9f, 0.3f);
        var sealLabel = MakeText(go.transform, "   ", 34, new Vector2(0, 150), TextAnchor.MiddleCenter, 300, 50);
        sealLabel.color = PalGold;
        sealLabel.gameObject.SetActive(false);
        var shieldLabel = MakeText(go.transform, "    ", 34, new Vector2(0, 190), TextAnchor.MiddleCenter, 280, 50);
        shieldLabel.color = PalBlue;
        shieldLabel.gameObject.SetActive(false);
        return new EnemyVisual
        {
            Go = go,
            Rt = rt,
            Fill = hp,
            SealLabel = sealLabel,
            ShieldLabel = shieldLabel.gameObject,
```

```
            Pos = e.Pos,
            Row = e.Row,
            Hp = e.Hp,
            MaxHp = e.MaxHp,
        };
    }
    // ——— HUD    ———
    public void SetFloor(int f) =&gt; _floorText.text = " " + f + " ";
    public void SetCharms(int n) =&gt; _charmText.text = "    ×" + n;
    public void SetKills(int k) =&gt; _killsText.text = "   " + k;
    public void SetHint(string hint) =&gt; _hintText.text = hint;
    public void HideIntro() =&gt; _introOverlay.gameObject.SetActive(false);
    public void ShowWallBanner(bool show) =&gt; _wallBanner.gameObject.SetActive(show);
    public void ShowFormationPanel(bool show) =&gt; _formationPanel.gameObject.SetActive(show);
    public void ShowFloorSkip(bool show) =&gt; _skipBanner.gameObject.SetActive(show);
    public void SetCodex(int litCount)
    {
        for (int i = 0; i &lt; NewbieConfig.CodexSlots; i++)
        {
            bool lit = i &lt; litCount;
            _codexSlots[i].color = lit ? PalFrostedGold : new Color(0.16f, 0.16f, 0.18f);   //    ·
            _codexLabels[i].text = lit ? (i == 0 ? " " : " ") : "?";
            _codexLabels[i].color = lit ? PalBlack : new Color(0.45f, 0.45f, 0.48f);
        }
    }
    public void PlaySealAnim()
    {
        StartCoroutine(SealAnimRoutine());
    }
    IEnumerator SealAnimRoutine()
    {
        _sealOverlay.gameObject.SetActive(true);
        float t = 0f;
        bool flashed = false;
        while (t &lt; NewbieConfig.SealAnimSeconds)
        {
            t += Time.unscaledDeltaTime;
            float k = t / NewbieConfig.SealAnimSeconds;
            _magicRing.localRotation = Quaternion.Euler(0f, 0f, k * 540f);          //   1.5
            _magicRing.localScale = Vector3.one * Mathf.Lerp(1.3f, 0.55f, k);      //
            if (!flashed &amp;&amp; k &gt; 0.75f)
            {
                flashed = true;
                StartCoroutine(FlashRoutine());
            }
            yield return null;
        }
        _sealOverlay.gameObject.SetActive(false);
        _magicRing.localRotation = Quaternion.identity;
        _magicRing.localScale = Vector3.one;
```

```
}
IEnumerator FlashRoutine()
{
    float t = 0f;
    while (t &lt; 0.25f)
    {
        t += Time.unscaledDeltaTime;
        float a = Mathf.PingPong(t / 0.25f * 2f, 1f);
        _sealFlash.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0.75f * a);
        yield return null;
    }
    _sealFlash.color = new Color(PalGold.r, PalGold.g, PalGold.b, 0f);
}
public void PlaySealFail()
{
    StartCoroutine(FailRoutine());
}
IEnumerator FailRoutine()
{
    _failOverlay.gameObject.SetActive(true);
    yield return new WaitForSecondsRealtime(NewbieConfig.SealFailAnimSeconds);
    _failOverlay.gameObject.SetActive(false);
}
public void ShowSettle(int kills, int charms, int codex)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = $"              {codex}/{NewbieConfig.CodexSlots}·        \n      ×{charms}     +{Newbie
}
// ————    ————
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = color;
    return rt;
}
RectTransform MakeSlot(RectTransform parent, Vector2 pos, Color color)
{
    var go = new GameObject("Slot", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(220, 220);
```

```
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = new Color(color.r, color.g, color.b, 0.6f);
        return rt;
    }
    void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
    {
        var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(420, 120);
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = PalGreen;
        MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120);
        go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
    }
    static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent&lt;Text&gt;();
        t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = new Color(0.91f, 0.89f, 0.85f);
        t.supportRichText = true;
        return t;
    }
    static Sprite MakeSquareSprite()
    {
        var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
        var px = tex.GetPixels32();
        for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
        tex.SetPixels32(px);
        tex.Apply(false, false);
        var sprite = Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        return sprite;
    }
    // ─────        ─────
    sealed class EnemyVisual
    {
        public GameObject Go;
```

```
            public RectTransform Rt;
            public Image Fill;
            public Text SealLabel;
            public GameObject ShieldLabel;
            public bool LastSealReady;
            public bool LastShielded;
            public int LastAttempts;
            public float Pos;
            public int Row;
            public float Hp;
            public float MaxHp;
            public bool SealReady;
            public bool Shielded;
            public int Attempts;
        }
    }
}
// IdleSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P04.Newbie
{
    /// &lt;summary&gt;
    ///              C#       Play   Editor headless   NewbieFlowCheck
    /// X025        ·   V1.2
    /// ①        1-3   →      4-9   → 10    ——§4.1    "      "
    /// ②      §4.2-1     50% +   3          ——          ×2→
    /// ③     §4.3 IdleFormula   2/      ——
    ///       HP     +    +  Buff  10%
    /// &lt;/summary&gt;
    public sealed class IdleSim
    {
        public enum IdleState { Prep, Fighting, SealAnim, SealFailAnim, FloorSkip, WallDemo, Formation, BossFight, Settl
        public sealed class FriendInst
        {
            public int Id;
            public string Name;
            public int Row;
            public float Hp;
            public float MaxHp;
            public float Cooldown;
        }
        public sealed class EnemyInst
        {
            public int Id;
            public string Name;
            public int Row;
            public float Pos;        // 1.0=     → FriendEngageRange=
            public float Hp;
            public float MaxHp;
```

```
    public float Atk;
    public float Speed;
    public float Cooldown;
    public bool Collectible;
    public bool Boss;
    public bool Shielded;
    public bool SealReady;
    public bool Dead;
    public int SealAttempts;   //          3
}
public IdleState State { get; private set; } = IdleState.Prep;
readonly List&lt;FriendInst&gt; _friends = new();
readonly List&lt;EnemyInst&gt; _enemies = new();
readonly List&lt;EnemyInst&gt; _deadThisTick = new();
int _nextId = 1;
int _floor;                //   1..3  =10
int _floorSpawnIdx;
float _floorTime;           //
float _floorGapTimer;       //         &gt;0=
float _prepTimer;
float _sealTimer;
float _wallTimer;
float _skipTimer;
bool _firstSealDone;
EnemyInst _sealTarget;
EnemyInst _sealReadyEnemy;
int _attackRotate;
public IReadOnlyList&lt;FriendInst&gt; Friends =&gt; _friends;
public IReadOnlyList&lt;EnemyInst&gt; Enemies =&gt; _enemies;
public int Life =&gt; NewbieConfig.StartLife;
public int SealCharms { get; private set; } = NewbieConfig.StartSealCharms;
public int CodexLitCount { get; private set; }
public int Kills { get; private set; }
public int Seals { get; private set; }              //
public int SealFails { get; private set; }          //
public int SealRefunds { get; private set; }        //
public int PitySaves { get; private set; }          //
public int IdleIncomeGranted { get; private set; }  //
public bool FloorSkipShown { get; private set; }    //       4-9
public bool GudiaoJoined { get; private set; }      //
public float BattleTime { get; private set; }
public float YinBuff { get; private set; } = 1f;
public EnemyInst SealReadyEnemy =&gt; _sealReadyEnemy;
public bool AdPopupShown =&gt; false;                  //
/// &lt;summary&gt;     Prep=0   1-3     =9     =10 &lt;/summary&gt;
public int Floor
{
    get
    {
        if (State == IdleState.Prep) return 0;
```

```
        if (State == IdleState.FloorSkip) return NewbieConfig.FloorSkipTo;
        if (State == IdleState.Settled || State == IdleState.Lost) return NewbieConfig.WallFloor;
        if (State == IdleState.WallDemo || State == IdleState.Formation || State == IdleState.BossFight
            || State == IdleState.SealAnim || State == IdleState.SealFailAnim)
            return _firstSealDone ? NewbieConfig.WallFloor : _floor;
        return _floor;
    }
}
public event Action&lt;EnemyInst&gt; EnemyKilled;
public event Action&lt;EnemyInst&gt; SealEnabled;
public event Action&lt;EnemyInst&gt; Sealed;
public event Action&lt;EnemyInst, int&gt; SealFailed;      // ( ,     )
public event Action WallDemoShown;
public event Action FormationUnlocked;
public event Action GudiaoPlaced;
public event Action&lt;int&gt; CharmsChanged;
public event Action&lt;IdleState&gt; StateChanged;
public IdleSim()
{
    for (int i = 0; i &lt; NewbieConfig.FriendCount; i++)
    {
        _friends.Add(new FriendInst
        {
            Id = _nextId++,
            Name = NewbieConfig.FriendNames[i],
            Row = i,
            Hp = NewbieConfig.FriendHp,
            MaxHp = NewbieConfig.FriendHp,
        });
    }
}
public void Tick(float dt)
{
    switch (State)
    {
        case IdleState.Prep:
            _prepTimer += dt;
            if (_prepTimer &gt;= NewbieConfig.IntroSeconds) StartHunt();
            return;
        case IdleState.Fighting:
        case IdleState.BossFight:
            TickBattle(dt);
            return;
        case IdleState.SealAnim:
            _sealTimer -= dt;
            if (_sealTimer &lt;= 0f) FinishSeal();
            return;
        case IdleState.SealFailAnim:
            _sealTimer -= dt;
            if (_sealTimer &lt;= 0f) FinishSealFail();
```

```
            return;
        case IdleState.FloorSkip:
            BattleTime += dt;
            _skipTimer -= dt;
            if (_skipTimer &lt;= 0f) EnterWallDemo();
            return;
        case IdleState.WallDemo:
            BattleTime += dt;
            TickEnemiesMove(dt);
            _wallTimer -= dt;
            if (_wallTimer &lt;= 0f)
            {
                SetState(IdleState.Formation);
                FormationUnlocked?.Invoke();
            }
            return;
        // Formation                    ——
    }
}
void StartHunt()
{
    if (State != IdleState.Prep) return;
    BattleTime = 0f;
    BeginFloor(1);
}
void BeginFloor(int floor)
{
    _floor = floor;
    _floorSpawnIdx = 0;
    _floorTime = 0f;
    _floorGapTimer = 0f;
    SetState(IdleState.Fighting);
}
SpawnDef[] CurrentFloorSpawns() =&gt; NewbieConfig.FloorSpawns[_floor - 1];
void TickBattle(float dt)
{
    BattleTime += dt;
    if (State == IdleState.Fighting)
    {
        if (_floorGapTimer &gt; 0f)
        {
            _floorGapTimer -= dt;
            if (_floorGapTimer &lt;= 0f) BeginFloor(_floor + 1);
            return;
        }
        _floorTime += dt;
        TickFloorSpawns();
        //           +      3   ——  HP       FinishSeal
        if (_floorSpawnIdx &gt;= CurrentFloorSpawns().Length &amp;&amp; _enemies.Count == 0
            &amp;&amp; _floor &lt; NewbieConfig.TutorialFloors)
```

```
            {
                _floorGapTimer = NewbieConfig.FloorGapSeconds;
            }
        }
        TickEnemiesMove(dt);
        TickFriendsAttack(dt);
        TickEnemiesAttack(dt);
        RemoveDead();
    }
    void TickFloorSpawns()
    {
        var spawns = CurrentFloorSpawns();
        while (_floorSpawnIdx &lt; spawns.Length &amp;&amp; spawns[_floorSpawnIdx].At &lt;= _floorTime)
        {
            _enemies.Add(NewEnemy(spawns[_floorSpawnIdx], shielded: false));
            _floorSpawnIdx++;
        }
    }
    EnemyInst NewEnemy(SpawnDef def, bool shielded)
    {
        return new EnemyInst
        {
            Id = _nextId++,
            Name = def.Name,
            Row = def.Row,
            Pos = 1f,
            Hp = def.Hp,
            MaxHp = def.Hp,
            Atk = def.Atk,
            Speed = def.Speed,
            Cooldown = def.Atk &gt; 0f ? 1f : 0f,
            Collectible = def.Collectible,
            Boss = def.Boss,
            Shielded = shielded,
        };
    }
    void TickEnemiesMove(float dt)
    {
        for (int i = 0; i &lt; _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (e.Pos &gt; NewbieConfig.FriendEngageRange)
                e.Pos = Math.Max(NewbieConfig.FriendEngageRange, e.Pos - e.Speed * dt);
        }
    }
    void TickFriendsAttack(float dt)
    {
        for (int i = 0; i &lt; _friends.Count; i++)
        {
            var f = _friends[i];
```

```
            f.Cooldown -= dt;
            if (f.Cooldown &gt; 0f) continue;
            var target = PickTarget(f);
            if (target == null) { f.Cooldown = 0.05f; continue; }
            f.Cooldown = NewbieConfig.FriendAtkInterval;
            DamageEnemy(target, NewbieConfig.FriendAtk * YinBuff);
        }
    }
    EnemyInst PickTarget(FriendInst f)
    {
        //                          ——
        EnemyInst best = null;
        for (int i = 0; i &lt; _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (e.Dead || e.Pos &gt; NewbieConfig.FriendEngageRange) continue;
            if (e.Row == f.Row) { best = e; break; }
            if (best == null) best = e;
        }
        return best;
    }
    void DamageEnemy(EnemyInst e, float dmg)
    {
        if (e.Shielded) return;
        float lockHp = e.Collectible ? e.MaxHp * NewbieConfig.SealHpRatio : 0f;
        e.Hp -= dmg;
        if (e.Hp &gt; lockHp) return;
        if (!e.Collectible)
        {
            e.Dead = true;
            _deadThisTick.Add(e);
            return;
        }
        e.Hp = lockHp;                          //      HP
        if (e.SealReady) return;
        e.SealReady = true;
        _sealReadyEnemy = e;
        SealEnabled?.Invoke(e);
    }
    void TickEnemiesAttack(float dt)
    {
        for (int i = 0; i &lt; _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (e.Atk &lt;= 0f || e.Dead || e.Shielded) continue;
            if (e.Pos &gt; NewbieConfig.FriendEngageRange) continue;
            e.Cooldown -= dt;
            if (e.Cooldown &gt; 0f) continue;
            e.Cooldown = 1.2f;
            var f = _friends[_attackRotate % _friends.Count];
```

```
{
    PlayerPrefs.SetInt(BalanceKey, Balance);
    var sb = new StringBuilder();
    for (int i = 0; i &lt; RedeemedYaoIds.Count; i++)
    {
        if (i &gt; 0) sb.Append(',');
        sb.Append(RedeemedYaoIds[i]);
    }
    PlayerPrefs.SetString(RedeemedKey, sb.ToString());
    PlayerPrefs.Save();
}
public static int EarnFor(bool won, int waveReached, int kills)
{
    int earned = waveReached * 2 + Math.Max(0, kills) / (won ? 10 : 5);
    if (won) earned += 10;
    return earned;
}
public void Deposit(int charms)
{
    if (charms &lt;= 0) return;
    Balance += charms;
    Save();
}
public bool CanExchange =&gt; Balance &gt;= ExchangeCost;
/// &lt;summary&gt;        +          id -1=    /     &lt;/summary&gt;
public int TryExchange(int roll)
{
    if (!CanExchange) return -1;
    var pool = YaoCodex.UnsealedIds();
    if (pool.Length == 0) return -1;
    int id = pool[((roll % pool.Length) + pool.Length) % pool.Length];
    YaoCodex.Seal(id);
    RedeemedYaoIds.Add(id);
    Balance = 0;
    Save();
    return id;
}
/// &lt;summary&gt;                    =             &lt;/summary&gt;
public int TryExchangeFair()
{
    int n = YaoCodex.UnsealedIds().Length;
    if (!CanExchange || n &lt;= 0) return -1;
    int limit = FairLimit(n);
    int r;
    do { r = UnityEngine.Random.Range(0, int.MaxValue); } while (r &gt;= limit);
    return TryExchange(r);
}
/// &lt;summary&gt;       limit   n      → roll%n       &lt;/summary&gt;
public static int FairLimit(int n)
{
```

```
            return int.MaxValue / n * n;
        }
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(BalanceKey);
            PlayerPrefs.DeleteKey(RedeemedKey);
            YaoCodex.ClearForTests();
        }
    }
}
// SealDisclosure.cs
using System.Collections.Generic;
using System.Globalization;
using UnityEngine;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    ///          01  V1.3       +U149     +X435
    ///   =   60       1                       0.01%
    ///                     disclosed   = P04SystemsCheck seal_disclosure
    ///                SealCharmWallet.TryExchangeFair
    ///   12 PlayerPrefs=
    /// &lt;/summary&gt;
    public static class SealDisclosure
    {
        /// &lt;summary&gt;01  cfg_gacha.disclosed    ——  true              &lt;/summary&gt;
        public const bool Disclosed = true;
        /// &lt;summary&gt;                        &lt;/summary&gt;
        public const int Cost = SealCharmWallet.ExchangeCost;
        const string SeenKey = "p04_disclosure_seen";
        /// &lt;summary&gt;           ·    0.01%·            &lt;/summary&gt;
        public static string PercentFor(int unsealedCount)
        {
            if (unsealedCount &lt;= 0) return "0.00%";
            return (100.0 / unsealedCount).ToString("F2", CultureInfo.InvariantCulture) + "%";
        }
        /// &lt;summary&gt;          +           =      &lt;/summary&gt;
        public static string[] Lines()
        {
            int[] pool = YaoCodex.UnsealedIds();
            var list = new List&lt;string&gt;
            {
                "        " + Cost + "        1      ",
                "         .   .     "
            };
            if (pool.Length == 0)
            {
                list.Add("             ");
                return list.ToArray();
            }
```

```
            string p = PercentFor(pool.Length);
            for (int i = 0; i &lt; pool.Length; i++)
            {
                var y = YaoCodex.ById(pool[i]);
                list.Add((y != null ? y.Name : "?") + " " + p);
            }
            return list.ToArray();
        }
        /// &lt;summary&gt;          &lt;/summary&gt;
        public static string PopupText()
        {
            return string.Join("\n", Lines());
        }
        /// &lt;summary&gt;           =true  &lt;/summary&gt;
        public static bool NeedsPopup()
        {
            return PlayerPrefs.GetInt(SeenKey, 0) == 0;
        }
        /// &lt;summary&gt;                &lt;/summary&gt;
        public static void MarkSeen()
        {
            PlayerPrefs.SetInt(SeenKey, 1);
            PlayerPrefs.Save();
        }
        public static void ClearForTests()
        {
            PlayerPrefs.DeleteKey(SeenKey);
        }
    }
}
// YaoCodex.cs
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    ///      V1.4     ·     ·              /
    /// =                              12 PlayerPrefs
    /// &lt;/summary&gt;
    public static class YaoCodex
    {
        public enum Tier { Common, Purple, Gold }
        public sealed class YaoDef
        {
            public int Id;
            public string Name;      //
            public string Role;      //
            public Tier Tier;
            public float Range;
```

```
    public float Atk;
    public float Interval;
    public bool Aoe;
    public string SealHint;
    public YaoDef(int id, string name, string role, Tier tier,
        float range, float atk, float interval, bool aoe, string sealHint)
    {
        Id = id; Name = name; Role = role; Tier = tier;
        Range = range; Atk = atk; Interval = interval; Aoe = aoe;
        SealHint = sealHint;
    }
}
static readonly YaoDef[] Yaos =
{
    new YaoDef(1, "   ", "    ", Tier.Common, 280f, 12f, 0.8f, true,  "   "),
    new YaoDef(2, "   ", "    ", Tier.Common, 520f, 26f, 1.1f, false, "   "),
    new YaoDef(3, "  ",   "    ", Tier.Purple, 480f, 10f, 1.2f, true,  "  "),
    new YaoDef(4, "  ",   "    ", Tier.Purple, 360f, 34f, 1.0f, false, "  "),
    new YaoDef(5, "   ", "    ", Tier.Purple, 420f, 14f, 1.1f, false, "  "),
    new YaoDef(6, "  ",   "    ", Tier.Gold, 300f, 40f, 1.3f, true,   "    "),
    new YaoDef(7, "  ",   "    ", Tier.Gold, 460f, 18f, 0.5f, false,  "    "),
    new YaoDef(8, "  ",   "    ", Tier.Gold, 520f, 30f, 1.2f, false,  "    "),
};
const string SealedKey = "p04_codex_sealed";
public static IReadOnlyList&lt;YaoDef&gt; All =&gt; Yaos;
public static YaoDef ById(int id)
{
    for (int i = 0; i &lt; Yaos.Length; i++)
        if (Yaos[i].Id == id) return Yaos[i];
    return null;
}
static HashSet&lt;int&gt; LoadSealed()
{
    var set = new HashSet&lt;int&gt;();
    string raw = PlayerPrefs.GetString(SealedKey, "");
    if (raw.Length == 0)
    {
        set.Add(1);
        set.Add(2);
        return set;
    }
    foreach (var part in raw.Split(','))
        if (int.TryParse(part, out int id)) set.Add(id);
    return set;
}
static void SaveSealed(HashSet&lt;int&gt; set)
{
    var sb = new System.Text.StringBuilder();
    int n = 0;
    foreach (int id in set)
```

```
            {
                if (n++ &gt; 0) sb.Append(',');
                sb.Append(id);
            }
            PlayerPrefs.SetString(SealedKey, sb.ToString());
            PlayerPrefs.Save();
        }
        public static bool IsSealed(int id) =&gt; LoadSealed().Contains(id);
        /// &lt;summary&gt;     ·  +             false=    &lt;/summary&gt;
        public static bool Seal(int id)
        {
            var set = LoadSealed();
            if (!set.Add(id)) return false;
            SaveSealed(set);
            return true;
        }
        public static List&lt;YaoDef&gt; Sealed()
        {
            var set = LoadSealed();
            var list = new List&lt;YaoDef&gt;();
            for (int i = 0; i &lt; Yaos.Length; i++)
                if (set.Contains(Yaos[i].Id)) list.Add(Yaos[i]);
            return list;
        }
        /// &lt;summary&gt;          · =      &lt;/summary&gt;
        public static int[] UnsealedIds()
        {
            var set = LoadSealed();
            var ids = new List&lt;int&gt;();
            for (int i = 0; i &lt; Yaos.Length; i++)
                if (!set.Contains(Yaos[i].Id)) ids.Add(Yaos[i].Id);
            return ids.ToArray();
        }
        public static void ClearForTests() =&gt; PlayerPrefs.DeleteKey(SealedKey);
    }
}
// YaotideBattleLaunch.cs
using UnityEngine;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    ///                ·   P03 BattleLaunch       →
    /// S04_Yaotide   YaotideLauncher        =
    /// &lt;/summary&gt;
    public static class YaotideBattleLaunch
    {
        public enum Mode { Random, DailyRift, FromCode, Tutorial }
        public static Mode Next = Mode.Random;
        public static int NextSeed;
        /// &lt;summary&gt;TD       T025b·X2366         TdTutorialData ·       &lt;/summary&gt;
```

```
        public static void StartTutorial()
        {
            Next = Mode.Tutorial;
            NextSeed = Newbie.TdTutorialData.TutorialSeed;
        }
        public static void StartRandom()
        {
            Next = Mode.Random;
            NextSeed = Mathf.Abs(UnityEngine.Random.Range(int.MinValue, int.MaxValue)) % 100000 + 1;
        }
        public static void StartDailyRift()
        {
            Next = Mode.DailyRift;
            NextSeed = DailyRift.SeedForToday();
        }
        public static bool StartFromFormationCode(string code)
        {
            var plan = YaotideFormationCodec.TryDecode(code);
            if (plan == null) return false;
            Next = Mode.FromCode;
            NextSeed = plan.Value.Seed;
            return true;
        }
        public static (Mode mode, int seed) Take()
        {
            var taken = (Next, NextSeed);
            Next = Mode.Random;
            NextSeed = 0;
            return taken;
        }
    }
}
// YaotideFormationCodec.cs
using System;
using System.Collections.Generic;
using System.Text;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    ///        P03 FormationCodec       +         base36+
    ///                      3bit+  1bit
    /// &lt;/summary&gt;
    public static class YaotideFormationCodec
    {
        const string Alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        const int Version = 1;
        public static string Encode(int seed, IReadOnlyList&lt;YaotideSim.TowerInst&gt; towers)
        {
            var sb = new StringBuilder();
            sb.Append(Digit(Version));
```

```
    AppendUnsigned(sb, seed &gt; 0 ? (uint)seed : 0u, 6);
    sb.Append(Digit(towers.Count));
    for (int i = 0; i &lt; towers.Count; i++)
        sb.Append(Digit(((towers[i].Slot &amp; 0x7) &lt;&lt; 1) | (towers[i].IsAoE ? 1 : 0)));
    sb.Append(ChecksumChar(sb.ToString()));
    return sb.ToString();
}
public static string Encode(YaotideSim sim)
    =&gt; Encode(sim.Seed, sim.Towers);
public readonly struct FormationPlan
{
    public readonly int Seed;
    public readonly int[] Slots;
    public readonly bool[] IsAoE;
    public FormationPlan(int seed, int[] slots, bool[] isAoE)
    {
        Seed = seed;
        Slots = slots;
        IsAoE = isAoE;
    }
}
public static FormationPlan? TryDecode(string code)
{
    if (string.IsNullOrEmpty(code) || code.Length &lt; 9) return null;
    string body = code.Substring(0, code.Length - 1);
    if (code[code.Length - 1] != ChecksumChar(body)) return null;
    int at = 0;
    if (Value(body[at++]) != Version) return null;
    uint seed = ReadUnsigned(body, ref at, 6);
    int count = Value(body[at++]);
    if (count &lt; 0 || count &gt; 8) return null;
    if (body.Length - at != count) return null;
    var slots = new int[count];
    bool[] aoe = new bool[count];
    for (int i = 0; i &lt; count; i++)
    {
        int cell = Value(body[at++]);
        if (cell &lt; 0) return null;
        slots[i] = (cell &gt;&gt; 1) &amp; 0x7;
        aoe[i] = (cell &amp; 1) == 1;
    }
    return new FormationPlan((int)seed, slots, aoe);
}
static char Digit(int v) =&gt; Alphabet[v % 36];
static int Value(char c) =&gt; Alphabet.IndexOf(char.ToUpperInvariant(c));
static void AppendUnsigned(StringBuilder sb, uint v, int chars)
{
    char[] buf = new char[chars];
    uint x = v;
    for (int i = chars - 1; i &gt;= 0; i--)
```

```
            {
                buf[i] = Alphabet[(int)(x % 36u)];
                x /= 36u;
            }
            for (int i = 0; i &lt; chars; i++) sb.Append(buf[i]);
        }
        static uint ReadUnsigned(string s, ref int at, int chars)
        {
            uint v = 0;
            for (int i = 0; i &lt; chars &amp;&amp; at &lt; s.Length; i++)
                v = v * 36u + (uint)Math.Max(0, Value(s[at++]));
            return v;
        }
        static char ChecksumChar(string body)
        {
            int sum = 0;
            for (int i = 0; i &lt; body.Length; i++) sum += body[i];
            return Alphabet[sum % 36];
        }
    }
}
// YaotideLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    /// S04_Yaotide         P03 BattleLauncher        YaotideSim+YaotideView
    ///     Tick    evt_rift   =RunResult   +     +   KV
    /// HUD           6
    /// &lt;/summary&gt;
    public sealed class YaotideLauncher : MonoBehaviour
    {
        YaotideSim _sim;
        YaotideView _view;
        Newbie.TdTutorialFlow _tutorial;
        Sy.Telemetry.TelemetryService _telemetry;
        bool _isTutorial;
        int _lastWave = -1, _lastLives = -1, _lastCharms = -1, _lastKills = -1;
        void Awake()
        {
            Application.targetFrameRate = 30;
            var canvas = FindObjectOfType&lt;Canvas&gt;();
            var (mode, seed) = YaotideBattleLaunch.Take();
            bool isRift = mode == YaotideBattleLaunch.Mode.DailyRift;
            _isTutorial = mode == YaotideBattleLaunch.Mode.Tutorial;
            _sim = _isTutorial
                ? new YaotideSim(Newbie.TdTutorialData.BuildLevel(), false)
                : new YaotideSim(YaotideLevel.Generate(seed), isRift);
            _view = gameObject.AddComponent&lt;YaotideView&gt;();
```

```
    _view.Initialize(canvas, _sim);
    _telemetry = new Sy.Telemetry.TelemetryService();
    _telemetry.AddSink(new TelemetryDebugSink());
    if (isRift) _telemetry.Track("evt_rift", DailyRift.EntryFields("entry_click"));
    _view.SlotTapped += slot =&gt; _sim.PlaceTower(slot);
    _view.ContractPicked += index =&gt; _sim.PickContract(index);
    _view.ExecuteRequested += () =&gt; _sim.TryExecute();
    _view.EnemyTapped += id =&gt; _sim.TapFocus(id);
    _sim.TowerPlaced += (tower, slot) =&gt; _view.SetTower(slot, tower.IsAoE, tower.YaoId);
    _sim.ContractOffered += () =&gt; _view.ShowContractPanel(_sim.ContractEntries);
    _sim.ExecutedFired += () =&gt; _view.PlayExecute();
    _sim.YaoSealed += id =&gt; _view.SetHint(YaoCodex.ById(id) != null
        ? YaoCodex.ById(id).Name + "      "
        : "    ");
    _sim.StateChanged += OnStateChanged;
    // TD        T025b·X2366           ——
    //       hint       evt_newbie_step   idle
    if (_isTutorial)
    {
        _tutorial = new Newbie.TdTutorialFlow(
            _sim,
            hint =&gt;
            {
                Debug.Log("[TD_HINT] " + hint);
                _view.SetHint(hint);
            },
            step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
                new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
    }
}
void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Wave != _lastWave) { _lastWave = _sim.Wave; _view.SetWave(_lastWave); }
    if (_sim.Lives != _lastLives) { _lastLives = _sim.Lives; _view.SetLives(_lastLives); }
    if (_sim.Charms != _lastCharms) { _lastCharms = _sim.Charms; _view.SetCharms(_lastCharms); }
    if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
}
void OnStateChanged(YaotideSim.YaotideState state)
{
    switch (state)
    {
        case YaotideSim.YaotideState.Prep:
            _view.HideIntro();
            break;
        case YaotideSim.YaotideState.Won:
            Settle(true);
            break;
        case YaotideSim.YaotideState.Lost:
```

```
                    Settle(false);
                    break;
            }
        }
        void Settle(bool won)
        {
            var result = YaotideRunResult.Snapshot(_sim, won);
            SealCharmWallet.Load().Deposit(result.CharmsEarned);
            if (_isTutorial &amp;amp;&amp;amp; won) Newbie.TdTutorialData.MarkTutorialDone();
            DailyRift.TrySubmit(result,
                Sy.Core.ServiceLocator.TryGet(out Sy.Platform.IOpenDataBoard board) ? board : null);
            _telemetry.Track("evt_rift",
                DailyRift.RunEndFields(result.Seed, result.Won, result.WaveReached));
            _view.ShowSettle(result);
        }
        sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
        {
            public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
            {
                var sb = new System.Text.StringBuilder(evtId);
                foreach (var kv in fields)
                    sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
                Debug.Log("[TELEMETRY] " + sb);
            }
        }
    }
}
// YaotideLevel.cs
using System;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    ///         V1.4    ·CEO   O-20261010-1245·    P03 SiegeLevel
    ///              ——
    ///   =    /  =    /  =  +2/  =     /  =
    ///   12     ——
    /// &lt;/summary&gt;
    public sealed class YaotideLevel
    {
        public const int WaveWindowMin = 5;
        public const int WaveWindowMax = 8;
        public readonly int Seed;
        public readonly int TotalWaves;
        public readonly float HpBase;
        public readonly float HpGrowth;
        public readonly float SpeedBase;
        public readonly float SpeedGrow;
        public readonly int CountBase;
        public readonly float SpawnInt;
        public readonly float WaveGap;
```

```
    public readonly int StartCharms;
    public readonly bool MidElite;
    public readonly string CalamityName;
    public readonly int CalamityId;
    public readonly float CalamityPower;
    YaotideLevel(int seed, System.Random r, int waveMin, int waveMax)
    {
        Seed = seed;
        TotalWaves = r.Next(waveMin, waveMax + 1);
        HpBase = Lerp(r, 40f, 58f);
        HpGrowth = Lerp(r, 1.18f, 1.28f);
        SpeedBase = Lerp(r, 72f, 88f);
        SpeedGrow = Lerp(r, 1.5f, 3.5f);
        CountBase = r.Next(7, 10);
        SpawnInt = Lerp(r, 0.6f, 1.0f);
        WaveGap = Lerp(r, 1.6f, 2.6f);
        StartCharms = r.Next(12, 20);
        MidElite = r.NextDouble() &lt; 0.5;
        CalamityId = r.Next(0, 5);
        CalamityPower = Lerp(r, 0.10f, 0.25f);
        CalamityName = CalamityId switch
        {
            0 =&gt; "  ",
            1 =&gt; "  ",
            2 =&gt; "  ",
            3 =&gt; "  ",
            _ =&gt; "  ",
        };
    }
    public static YaotideLevel Generate(int seed)
        =&gt; Generate(seed, WaveWindowMin, WaveWindowMax);
    public static YaotideLevel Generate(int seed, int waveMin, int waveMax)
        =&gt; new YaotideLevel(seed, new System.Random(seed), waveMin, waveMax);
    public WaveDef GetWave(int w)
    {
        float hp = HpBase * (float)Math.Pow(HpGrowth, w - 1)
                   * (CalamityId == 1 ? 1f + CalamityPower : 1f);   //
        int count = CountBase + (w / 3) + (CalamityId == 2 ? 2 : 0);   //
        float speed = SpeedBase + SpeedGrow * w + (CalamityId == 0 ? SpeedBase * CalamityPower : 0f);
        bool boss = w == TotalWaves || (MidElite &amp;&amp; w == TotalWaves - 1 &amp;&amp; w &gt;= 3);
        return new WaveDef(w, count, hp, speed, boss);
    }
    static float Lerp(System.Random r, float min, float max)
        =&gt; min + (float)r.NextDouble() * (max - min);
}
public readonly struct WaveDef
{
    public readonly int Wave;
    public readonly int Count;
    public readonly float Hp;
```

```
        public readonly float Speed;
        public readonly bool Boss;
        public WaveDef(int wave, int count, float hp, float speed, bool boss)
        {
            Wave = wave;
            Count = count;
            Hp = hp;
            Speed = speed;
            Boss = boss;
        }
    }
}
// YaotideRunResult.cs
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;          P03 RunResult                        &lt;/summary&gt;
    public sealed class YaotideRunResult
    {
        public bool Won;
        public int Seed;
        public bool IsDailyRift;
        public int WaveReached;
        public int Kills;
        public int LivesLeft;
        public int CharmsLeft;
        public int CharmsEarned;
        public float DurationSec;
        public string FormationCode;
        public string CalamityName;
        public int SealedYaoId;      //        -1=
        public static YaotideRunResult Snapshot(YaotideSim sim, bool won)
        {
            return new YaotideRunResult
            {
                Won = won,
                Seed = sim.Seed,
                IsDailyRift = sim.IsDailyRift,
                WaveReached = sim.Wave,
                Kills = sim.Kills,
                LivesLeft = sim.Lives,
                CharmsLeft = sim.Charms,
                DurationSec = sim.BattleTime,
                FormationCode = YaotideFormationCodec.Encode(sim),
                CalamityName = sim.Level != null ? sim.Level.CalamityName : "",
                CharmsEarned = SealCharmWallet.EarnFor(won, sim.Wave, sim.Kills),
                SealedYaoId = sim.SealedThisRun,
            };
        }
    }
}
```

```
// YaotideSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    ///          C#      ·V1.4     TD·CEO   O-20261010-1245
    /// S         A*·AI      +8   +    +     +
    ///         +           ≤40%     +       =
    ///     =            /     ·
    ///   12
    /// &lt;/summary&gt;
    public sealed class YaotideSim
    {
        public enum YaotideState { Intro, Prep, Fighting, ContractPause, Won, Lost }
        public sealed class TowerInst
        {
            public int Id;
            public int Slot;
            public bool IsAoE;
            public int YaoId;
            public float Range, Atk, Interval, Cooldown;
        }
        public sealed class EnemyInst
        {
            public int Id;
            public bool Boss;      //
            public bool Elite;     //
            public int Seg;
            public float SegT;
            public float X, Y;
            public float Hp, MaxHp;
            public float Speed;
            public bool Dead;
        }
        public YaotideState State { get; private set; } = YaotideState.Intro;
        public YaotideLevel Level { get; }
        public int Seed { get; }
        public bool IsDailyRift { get; private set; }
        public int TotalWaves =&gt; Level.TotalWaves;
        /// &lt;summary&gt;           RunResult   -1=   &lt;/summary&gt;
        public int SealedThisRun { get; private set; } = -1;
        // S       8   7       →       8
        public static readonly float[] PathX = { -460f, 340f, 340f, -340f, -340f, 340f, 340f, -500f };
        public static readonly float[] PathY = { 740f, 740f, 300f, 300f, -140f, -140f, -580f, -580f };
        public static readonly float[] SlotX = { 0f, 0f, 0f, -320f, 320f, -320f, 320f, -420f };
        public static readonly float[] SlotY = { 620f, 420f, -280f, 620f, 420f, -280f, 620f, -280f };
        public const float IntroSeconds = 3f;
        public const float PrepStartSeconds = 15f;
        public const float FocusSeconds = 4f;
```

```
public const float BossHpBase = 900f;
public const float BossSpeed = 55f;
public const int ExecuteHpRatioPermille = 400;   // ≤40%
public const int KillCharm = 2;
public const int WaveClearCharm = 8;
readonly List&lt;TowerInst&gt; _towers = new();
readonly List&lt;EnemyInst&gt; _enemies = new();
readonly float[] _segLens;
int _nextId = 1;
float _stateTimer;
int _placed;
int _waveSpawned;
float _spawnTimer;
bool _spawning;
float _waveGapTimer;
int _lastCleared = -1;
int _contractCount;
readonly int[] _contractPool;      //
int _focusId = -1;
float _focusTimer;
float _enemySpeedMul = 1f;
float _atkMul = 1f;
float _towerIntervalMul = 1f;
float _charmMul = 1f;
float _fireMul = 1f;               //
int _waveBonus;
readonly int _eliteYaoId;
public IReadOnlyList&lt;TowerInst&gt; Towers =&gt; _towers;
public IReadOnlyList&lt;EnemyInst&gt; Enemies =&gt; _enemies;
public int Wave { get; private set; }
public int Lives { get; private set; } = 3;
public int Charms { get; private set; }
public int Kills { get; private set; }
public int PlacedCount =&gt; _placed;
public float BattleTime { get; private set; }
public event Action&lt;TowerInst, int&gt; TowerPlaced;
public event Action&lt;int&gt; WaveStarted;
public event Action&lt;int&gt; WaveCleared;
public event Action&lt;EnemyInst&gt; EnemySpawned;
public event Action&lt;EnemyInst&gt; EnemyKilled;
public event Action&lt;int&gt; LifeLost;
public event Action ContractOffered;
public event Action&lt;int&gt; ContractPicked;
public event Action BossKilled;
public event Action ExecutedFired;
public event Action&lt;int&gt; YaoSealed;
public event Action&lt;YaotideState&gt; StateChanged;
public int[] ContractEntries =&gt; _contractPool;
public YaotideSim(YaotideLevel level, bool isDailyRift)
{
```

```
        Level = level ?? throw new ArgumentNullException(nameof(level));
        IsDailyRift = isDailyRift;
        Seed = level.Seed;
        Charms = level.StartCharms;
        _segLens = new float[PathX.Length - 1];
        for (int s = 0; s &lt; _segLens.Length; s++)
        {
            float dx = PathX[s + 1] - PathX[s];
            float dy = PathY[s + 1] - PathY[s];
            _segLens[s] = (float)Math.Sqrt(dx * dx + dy * dy);
        }
        if (level.CalamityId == 3) _charmMul = 1f + level.CalamityPower;        //
        if (level.CalamityId == 4) _towerIntervalMul = 1f + level.CalamityPower; //
        var rng = new System.Random(level.Seed ^ 0x5A11);
        var bag = new List&lt;int&gt; { 0, 1, 2, 3, 4, 5 };
        _contractPool = new int[3];
        for (int i = 0; i &lt; 3; i++)
        {
            int at = rng.Next(bag.Count);
            _contractPool[i] = bag[at];
            bag.RemoveAt(at);
        }
        _eliteYaoId = 3 + level.Seed % 3;   //   /  /
    }
    (int Count, float Hp, float Speed, bool Boss, bool Elite) WaveAt(int w)
    {
        var def = Level.GetWave(w);
        float hp = def.Hp * _fireMul;
        _fireMul = 1f;
        bool final = w == Level.TotalWaves;
        bool elite = def.Boss &amp;&amp; !final;
        return (def.Count, hp, def.Speed, final, elite);
    }
    /// &lt;summary&gt;      +             +      &lt;/summary&gt;
    public bool PlaceTower(int slot)
    {
        if (State != YaotideState.Prep &amp;&amp; State != YaotideState.Fighting) return false;
        if (slot &lt; 0 || slot &gt;= SlotX.Length) return false;
        for (int i = 0; i &lt; _towers.Count; i++)
            if (_towers[i].Slot == slot) return false;
        var sealedList = YaoCodex.Sealed();
        if (sealedList.Count == 0) return false;
        var def = sealedList[_placed % sealedList.Count];
        int cost = CostFor(def.Tier);
        if (Charms &lt; cost) return false;
        Charms -= cost;
        var tower = new TowerInst
        {
            Id = _nextId++,
            Slot = slot,
```

```
            IsAoE = def.Aoe,
            YaoId = def.Id,
            Range = def.Range,
            Atk = def.Atk,
            Interval = def.Interval,
        };
        _towers.Add(tower);
        _placed++;
        TowerPlaced?.Invoke(tower, slot);
        return true;
    }
    static int CostFor(YaoCodex.Tier tier) =&gt; tier switch
    {
        YaoCodex.Tier.Purple =&gt; 9,
        YaoCodex.Tier.Gold =&gt; 12,
        _ =&gt; 6,
    };
    public void TapFocus(int enemyId)
    {
        for (int i = 0; i &lt; _enemies.Count; i++)
            if (_enemies[i].Id == enemyId)
            {
                _focusId = enemyId;
                _focusTimer = FocusSeconds;
                return;
            }
    }
    public bool IsFocused(EnemyInst e) =&gt; e.Id == _focusId &amp;&amp; _focusTimer &gt; 0f;
    /// &lt;summary&gt;      ≤40%↛     &lt;/summary&gt;
    public bool ExecuteReady
    {
        get
        {
            for (int i = 0; i &lt; _enemies.Count; i++)
            {
                var e = _enemies[i];
                if (e.Boss &amp;&amp; e.Hp * 1000f &lt;= e.MaxHp * ExecuteHpRatioPermille) return true;
            }
            return false;
        }
    }
    /// &lt;summary&gt;               &lt;/summary&gt;
    public bool TryExecute()
    {
        if (State != YaotideState.Fighting || !ExecuteReady) return false;
        for (int i = 0; i &lt; _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (!e.Boss) continue;
            ExecutedFired?.Invoke();
```

```
            DamageEnemy(e, e.Hp + 1f);
            return true;
        }
        return false;
    }
    /// &lt;summary&gt;          0-5       &lt;/summary&gt;
    public void PickContract(int index)
    {
        if (State != YaotideState.ContractPause) return;
        if (index &lt; 0 || index &gt;= _contractPool.Length) return;
        int entry = _contractPool[index];
        switch (entry)
        {
            case 0: _enemySpeedMul = 0.8f; break;                    //
            case 1: _atkMul = 1.25f; break;                          //
            case 2: _waveBonus = 10; break;                          //
            case 3: Charms += (int)(15 * _charmMul); break;          //
            case 4: Lives += 1; break;                               //
            case 5: _fireMul = 0.85f; break;                          //
        }
        _waveGapTimer = 0f;
        ContractPicked?.Invoke(entry);
        SetState(YaotideState.Fighting);
    }
    public void Tick(float dt)
    {
        switch (State)
        {
            case YaotideState.Intro:
                _stateTimer += dt;
                if (_stateTimer &gt;= IntroSeconds) SetState(YaotideState.Prep);
                return;
            case YaotideState.Prep:
                _stateTimer += dt;
                if (_placed &gt;= 1 &amp;&amp; (_stateTimer &gt;= PrepStartSeconds))
                {
                    SetState(YaotideState.Fighting);
                    BeginWave(1);
                }
                return;
            case YaotideState.Fighting:
                BattleTime += dt;
                if (_focusTimer &gt; 0f) _focusTimer -= dt;
                TickSpawns(dt);
                TickEnemies(dt);
                TickTowers(dt);
                TickWaves(dt);
                if (Lives &lt;= 0) SetState(YaotideState.Lost);
                return;
        }
```

```
}
void BeginWave(int w)
{
    Wave = w;
    _waveSpawned = 0;
    _spawnTimer = 0f;
    _waveGapTimer = 0f;
    _spawning = true;
    WaveStarted?.Invoke(w);
    float bossHp = BossHpBase * (1f + 0.12f * Level.TotalWaves);
    var (_, _, _, boss, elite) = WaveAt(w);
    if (boss || elite)
    {
        var e = new EnemyInst
        {
            Id = _nextId++,
            Boss = boss,
            Elite = elite,
            Seg = 0,
            SegT = 0f,
            X = PathX[0],
            Y = PathY[0],
            Hp = boss ? bossHp : bossHp * 0.55f,
            MaxHp = boss ? bossHp : bossHp * 0.55f,
            Speed = elite ? BossSpeed * 1.25f : BossSpeed,
        };
        _enemies.Add(e);
        EnemySpawned?.Invoke(e);
    }
}
void TickSpawns(float dt)
{
    if (!_spawning) return;
    var (count, hp, speed, _, _) = WaveAt(Wave);
    _spawnTimer += dt;
    while (_spawnTimer &gt;= Level.SpawnInt &amp;&amp; _waveSpawned &lt; count)
    {
        _spawnTimer -= Level.SpawnInt;
        _waveSpawned++;
        var e = new EnemyInst
        {
            Id = _nextId++,
            Seg = 0,
            SegT = 0f,
            X = PathX[0],
            Y = PathY[0],
            Hp = hp,
            MaxHp = hp,
            Speed = speed,
        };
```

```
            _enemies.Add(e);
            EnemySpawned?.Invoke(e);
        }
        if (_waveSpawned &gt;= count) _spawning = false;
    }
    void TickEnemies(float dt)
    {
        for (int i = _enemies.Count - 1; i &gt;= 0; i--)
        {
            var e = _enemies[i];
            float move = e.Speed * _enemySpeedMul * dt;
            while (move &gt; 0f)
            {
                float segLen = _segLens[e.Seg];
                float remain = (1f - e.SegT) * segLen;
                if (move &lt; remain)
                {
                    e.SegT += move / segLen;
                    move = 0f;
                }
                else
                {
                    move -= remain;
                    e.Seg++;
                    e.SegT = 0f;
                    if (e.Seg &gt;= _segLens.Length)
                    {
                        //              =
                        if (e.Boss)
                        {
                            e.Seg = 0;
                            e.SegT = 0f;
                        }
                        else
                        {
                            _enemies.RemoveAt(i);
                        }
                        Lives = Math.Max(0, Lives - 1);
                        LifeLost?.Invoke(Lives);
                        break;
                    }
                }
            }
            if (e.Seg &lt; _segLens.Length)
            {
                e.X = Lerp(PathX[e.Seg], PathX[e.Seg + 1], e.SegT);
                e.Y = Lerp(PathY[e.Seg], PathY[e.Seg + 1], e.SegT);
            }
        }
    }
```

```
static float Lerp(float a, float b, float t) =&gt; a + (b - a) * t;
void TickTowers(float dt)
{
    for (int i = 0; i &lt; _towers.Count; i++)
    {
        var t = _towers[i];
        t.Cooldown -= dt;
        if (t.Cooldown &gt; 0f) continue;
        bool any = false;
        if (t.IsAoE)
        {
            float r2 = t.Range * t.Range;
            for (int j = _enemies.Count - 1; j &gt;= 0; j--)
            {
                var e = _enemies[j];
                float dx = e.X - SlotX[t.Slot], dy = e.Y - SlotY[t.Slot];
                if (dx * dx + dy * dy &gt; r2) continue;
                any = true;
                DamageEnemy(e, t.Atk * _atkMul);
            }
        }
        else
        {
            float r2 = t.Range * t.Range;
            EnemyInst target = null;
            float best = float.MaxValue;
            for (int j = 0; j &lt; _enemies.Count; j++)
            {
                var e = _enemies[j];
                float dx = e.X - SlotX[t.Slot], dy = e.Y - SlotY[t.Slot];
                float d2 = dx * dx + dy * dy;
                if (d2 &gt; r2) continue;
                if (IsFocused(e)) { target = e; best = -1f; break; }
                if (d2 &lt; best) { best = d2; target = e; }
            }
            if (target != null)
            {
                any = true;
                DamageEnemy(target, t.Atk * _atkMul);
            }
        }
        t.Cooldown = any ? t.Interval * _towerIntervalMul : 0.05f;
    }
}
void DamageEnemy(EnemyInst e, float dmg)
{
    e.Hp -= dmg;
    if (e.Hp &gt; 0f) return;
    e.Dead = true;
    _enemies.Remove(e);
```

```
        Kills++;
        Charms += (int)(KillCharm * _charmMul);
        if (e.Boss)
        {
            SealInto(_bossYaoId());
            //
            for (int i = _enemies.Count - 1; i &gt;= 0; i--)
            {
                var rest = _enemies[i];
                rest.Dead = true;
                _enemies.RemoveAt(i);
                Kills++;
                Charms += (int)(KillCharm * _charmMul);
                EnemyKilled?.Invoke(rest);
            }
            BossKilled?.Invoke();
            EnemyKilled?.Invoke(e);
            SetState(YaotideState.Won);
            return;
        }
        if (e.Elite)
        {
            SealInto(_eliteYaoId);
        }
        EnemyKilled?.Invoke(e);
    }
    int _bossYaoId() =&gt; 8;   //
    void SealInto(int yaoId)
    {
        SealedThisRun = yaoId;
        YaoCodex.Seal(yaoId);
        YaoSealed?.Invoke(yaoId);
    }
    void TickWaves(float dt)
    {
        if (_spawning || _enemies.Count &gt; 0) return;
        _waveGapTimer += dt;
        if (_waveGapTimer &lt; Level.WaveGap) return;
        int cleared = Wave;
        if (cleared &gt;= TotalWaves) return;   //
        if (_lastCleared != cleared)
        {
            _lastCleared = cleared;
            Charms += (int)((WaveClearCharm + _waveBonus) * _charmMul);
            WaveCleared?.Invoke(cleared);
        }
        if (_contractCount &lt; _contractPool.Length)
        {
            _contractCount++;
            SetState(YaotideState.ContractPause);
```

```
                ContractOffered?.Invoke();
                return;
            }
            BeginWave(cleared + 1);
        }
        void SetState(YaotideState s)
        {
            if (State == s) return;
            State = s;
            _stateTimer = 0f;
            StateChanged?.Invoke(s);
        }
    }
}
// YaotideView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P04.Yaotide
{
    /// &lt;summary&gt;
    ///             UGUI·        ×  ×  ×  ×
    ///   /  /  /  /HUD/     /    /
    /// HUD           6  30fps
    /// &lt;/summary&gt;
    public sealed class YaotideView : MonoBehaviour
    {
        Canvas _canvas;
        Sprite _square;
        Text _waveText, _livesText, _charmText, _killsText, _hintText, _calamityText;
        Button _executeButton;
        Image _executeImage;
        RectTransform _introOverlay, _contractPanel, _executeOverlay, _settlePanel;
        Text _settleStats, _settleTitle;
        int _totalWaves = YaotideLevel.WaveWindowMax;
        readonly Text[] _contractNames = new Text[3];
        readonly Text[] _contractDescs = new Text[3];
        readonly Dictionary&lt;int, EnemyVisual&gt; _enemies = new();
        readonly List&lt;int&gt; _scratch = new();
        readonly Image[] _slotImages = new Image[YaotideSim.SlotX.Length];
        readonly Text[] _slotLabels = new Text[YaotideSim.SlotX.Length];
        public event Action&lt;int&gt; SlotTapped;
        public event Action&lt;int&gt; EnemyTapped;
        public event Action&lt;int&gt; ContractPicked;
        public event Action ExecuteRequested;
        static readonly Color PalRed = new Color(0.902f, 0.224f, 0.275f);    // #E63946
        static readonly Color PalBlue = new Color(0.271f, 0.482f, 0.616f);   // #457B9D
        static readonly Color PalGold = new Color(1f, 0.843f, 0f);           // #FFD700
```

```
static readonly Color PalGreen = new Color(0.165f, 0.616f, 0.561f);  // #2A9D8F
static readonly Color PalBlack = new Color(0.114f, 0.114f, 0.114f);  // #1D1D1D
static readonly Color PathCol = new Color(0.16f, 0.10f, 0.10f);
/// &lt;summary&gt;    id 0-5   YaotideSim.PickContract     &lt;/summary&gt;
public static readonly (string Name, string Desc)[] ContractCards =
{
    ("   ", "    -20%"),
    ("   ", "  +25%"),
    ("   ", "     +10"),
    ("   ", "   +15"),
    ("   ", "  +1"),
    ("   ", "     -15%"),
};
public void Initialize(Canvas canvas, YaotideSim sim)
{
    _canvas = canvas;
    _square = MakeSquareSprite();
    _totalWaves = sim.TotalWaves;
    BuildBackdrop();
    BuildHud();
    BuildPath();
    BuildSlots();
    BuildIntroOverlay();
    BuildContractPanel();
    BuildExecuteOverlay();
    BuildSettlePanel();
    SetHint("    ——     ");
    if (sim.Level != null) SetCalamity("   · " + sim.Level.CalamityName);
}
void BuildBackdrop()
{
    var art = Sy.Core.ArtRuntime.Get("P04_scene_01");
    if (art == null) return;
    var go = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    rt.SetAsFirstSibling();
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = art;
    image.color = Color.white;
    image.preserveAspect = true;
    image.raycastTarget = false;
}
void BuildHud()
{
    var root = _canvas.transform;
    _waveText = MakeText(root, "   0/" + _totalWaves, 46, new Vector2(0, 880), TextAnchor.UpperLeft, 380, 70);
```

```
        _waveText.color = PalGold;
        _livesText = MakeText(root, "   3", 46, new Vector2(0, 880), TextAnchor.UpperRight, 340, 70);
        _charmText = MakeText(root, "  0", 44, new Vector2(0, 820), TextAnchor.UpperLeft, 380, 60);
        _charmText.color = PalGold;
        _killsText = MakeText(root, "   0", 44, new Vector2(0, 820), TextAnchor.UpperRight, 380, 60);
        _calamityText = MakeText(root, "", 36, new Vector2(0, 760), TextAnchor.UpperLeft, 520, 50);
        _calamityText.color = PalGreen;
        _hintText = MakeText(root, "", 40, new Vector2(0, -780), TextAnchor.MiddleCenter, 1020, 150);
        _hintText.color = new Color(0.95f, 0.87f, 0.6f);
        var btnGo = new GameObject("ExecuteButton", typeof(Image), typeof(Button));
        btnGo.transform.SetParent(root, false);
        var btnRt = (RectTransform)btnGo.transform;
        btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0f);
        btnRt.anchoredPosition = new Vector2(0, 120);
        btnRt.sizeDelta = new Vector2(460, 140);
        _executeImage = btnGo.GetComponent&lt;Image&gt;();
        _executeImage.sprite = _square;
        _executeImage.color = new Color(0.35f, 0.35f, 0.35f);
        _executeButton = btnGo.GetComponent&lt;Button&gt;();
        _executeButton.interactable = false;
        MakeText(btnGo.transform, "   ", 54, Vector2.zero, TextAnchor.MiddleCenter, 460, 140).color = PalBlack;
        _executeButton.onClick.AddListener(() =&gt; ExecuteRequested?.Invoke());
    }
    void BuildPath()
    {
        for (int s = 0; s &lt; YaotideSim.PathX.Length - 1; s++)
        {
            float x0 = YaotideSim.PathX[s], y0 = YaotideSim.PathY[s];
            float x1 = YaotideSim.PathX[s + 1], y1 = YaotideSim.PathY[s + 1];
            bool horiz = Math.Abs(y1 - y0) &lt; 1f;
            var go = new GameObject("PathSeg_" + s, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(_canvas.transform, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2((x0 + x1) * 0.5f, (y0 + y1) * 0.5f);
            rt.sizeDelta = horiz
                ? new Vector2(Math.Abs(x1 - x0) + 90f, 90f)
                : new Vector2(90f, Math.Abs(y1 - y0) + 90f);
            var img = go.GetComponent&lt;Image&gt;();
            img.sprite = _square;
            img.color = PathCol;
        }
        MakeText(_canvas.transform, "    ", 34, new Vector2(-460f, 820f), TextAnchor.MiddleCenter, 260, 50).color =
        MakeText(_canvas.transform, "    ", 34, new Vector2(-500f, -660f), TextAnchor.MiddleCenter, 260, 50).color =
    }
    void BuildSlots()
    {
        for (int i = 0; i &lt; YaotideSim.SlotX.Length; i++)
        {
            int index = i;
```

```
            var go = new GameObject("Slot_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
            go.transform.SetParent(_canvas.transform, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(YaotideSim.SlotX[i], YaotideSim.SlotY[i]);
            rt.sizeDelta = new Vector2(140, 140);
            var img = go.GetComponent&lt;Image&gt;();
            var slotArt = Sy.Core.ArtRuntime.Get("P04_unit_01");
            if (slotArt != null)
            {
                img.sprite = slotArt;
                img.color = Color.white;
                img.preserveAspect = true;
            }
            else
            {
                img.sprite = _square;
                img.color = new Color(0.13f, 0.13f, 0.13f, 0.9f);
            }
            _slotImages[i] = img;
            _slotLabels[i] = MakeText(go.transform, "  ", 34, Vector2.zero, TextAnchor.MiddleCenter, 140, 140);
            _slotLabels[i].color = new Color(0.55f, 0.52f, 0.45f);
            go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; SlotTapped?.Invoke(index));
        }
    }
    void BuildIntroOverlay()
    {
        _introOverlay = MakeOverlayPanel("IntroOverlay", new Color(0.05f, 0.02f, 0.08f, 0.95f));
        MakeText(_introOverlay, "       ", 100, new Vector2(0, 100), TextAnchor.MiddleCenter, 1000, 130).color = Pal
        MakeText(_introOverlay, "      ·       ", 46, new Vector2(0, -60), TextAnchor.MiddleCenter, 1000, 80).color
    }
    void BuildContractPanel()
    {
        _contractPanel = MakeOverlayPanel("ContractPanel", new Color(0.03f, 0.03f, 0.06f, 0.92f));
        MakeText(_contractPanel, "   ·    ", 76, new Vector2(0, 560), TextAnchor.MiddleCenter, 1000, 100).color = Pa
        for (int i = 0; i &lt; 3; i++)
        {
            int index = i;
            var card = ContractCards[i];
            var go = new GameObject("Contract_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
            go.transform.SetParent(_contractPanel, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(-330 + i * 330, 0);
            rt.sizeDelta = new Vector2(300, 420);
            var img = go.GetComponent&lt;Image&gt;();
            img.sprite = _square;
            img.color = i == 0 ? new Color(0.55f, 0.12f, 0.15f) : new Color(0.14f, 0.16f, 0.20f);
            _contractNames[i] = MakeText(go.transform, " " + card.Name + " ", 56, new Vector2(0, 120), TextAnchor.Mi
            _contractNames[i].color = PalGold;
```

```
        _contractDescs[i] = MakeText(go.transform, card.Desc, 38, new Vector2(0, -40), TextAnchor.MiddleCenter,
        _contractDescs[i].color = new Color(0.85f, 0.85f, 0.8f);
        go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt;
        {
            _contractPanel.gameObject.SetActive(false);
            ContractPicked?.Invoke(index);
        });
    }
    MakeText(_contractPanel, "    ——    ", 40, new Vector2(0, -400), TextAnchor.MiddleCenter, 1000, 80).color =
    _contractPanel.gameObject.SetActive(false);
}
/// &lt;summary&gt;            &lt;/summary&gt;
public void ShowContractPanel(int[] entries)
{
    for (int i = 0; i &lt; 3 &amp;&amp; i &lt; entries.Length; i++)
    {
        var card = ContractCards[Math.Max(0, Math.Min(ContractCards.Length - 1, entries[i]))];
        _contractNames[i].text = " " + card.Name + " ";
        _contractDescs[i].text = card.Desc;
    }
    _contractPanel.gameObject.SetActive(true);
}
void BuildExecuteOverlay()
{
    _executeOverlay = MakeOverlayPanel("ExecuteOverlay", new Color(1f, 0.98f, 0.92f, 0.75f));
    var lineGo = new GameObject("ExecuteLine", typeof(RectTransform), typeof(Image));
    lineGo.transform.SetParent(_executeOverlay, false);
    var lineRt = (RectTransform)lineGo.transform;
    lineRt.anchorMin = lineRt.anchorMax = new Vector2(0.5f, 0.5f);
    lineRt.anchoredPosition = Vector2.zero;
    lineRt.sizeDelta = new Vector2(2200, 16);
    lineRt.localRotation = Quaternion.Euler(0f, 0f, 24f);
    var lineImg = lineGo.GetComponent&lt;Image&gt;();
    lineImg.sprite = _square;
    lineImg.color = PalRed;
    MakeText(_executeOverlay, "  ", 110, new Vector2(0, -180), TextAnchor.MiddleCenter, 1000, 140).color = PalBl
    _executeOverlay.gameObject.SetActive(false);
}
void BuildSettlePanel()
{
    _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.03f, 0.03f, 0.05f, 0.96f));
    _settleTitle = MakeText(_settlePanel, "    ", 90, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 110);
    _settleTitle.color = PalGold;
    _settleStats = MakeText(_settlePanel, "", 46, new Vector2(0, 380), TextAnchor.MiddleCenter, 1000, 260);
    MakeActionButton(_settlePanel, "    ", new Vector2(-260, -260), () =&gt; UnityEngine.SceneManagement.SceneMa
    MakeActionButton(_settlePanel, "     ", new Vector2(260, -260), () =&gt; UnityEngine.SceneManagement.SceneMa
    _settlePanel.gameObject.SetActive(false);
}
// ————      ————
public void SyncView(YaotideSim sim)
```

```
{
    ReconcileEnemies(sim);
    foreach (var kv in _enemies)
    {
        var v = kv.Value;
        v.Rt.anchoredPosition = new Vector2(v.X, v.Y);
        if (v.Fill != null) v.Fill.fillAmount = v.MaxHp &gt; 0f ? v.Hp / v.MaxHp : 0f;
        bool focused = v.Focused;
        if (focused != v.LastFocused)
        {
            v.LastFocused = focused;
            v.FocusLabel.gameObject.SetActive(focused);
        }
    }
    bool ready = sim.ExecuteReady;
    if (_executeButton.interactable != ready)
    {
        _executeButton.interactable = ready;
        _executeImage.color = ready ? PalRed : new Color(0.35f, 0.35f, 0.35f);
    }
}
void ReconcileEnemies(YaotideSim sim)
{
    _scratch.Clear();
    foreach (var kv in _enemies)
        if (!ContainsEnemy(sim, kv.Key)) _scratch.Add(kv.Key);
    for (int i = 0; i &lt; _scratch.Count; i++)
    {
        Destroy(_enemies[_scratch[i]].Go);
        _enemies.Remove(_scratch[i]);
    }
    for (int i = 0; i &lt; sim.Enemies.Count; i++)
    {
        var e = sim.Enemies[i];
        if (!_enemies.TryGetValue(e.Id, out var v))
        {
            v = CreateEnemyVisual(e);
            _enemies[e.Id] = v;
        }
        v.X = e.X;
        v.Y = e.Y;
        v.Hp = e.Hp;
        v.MaxHp = e.MaxHp;
        v.Focused = sim.IsFocused(e);
    }
}
static bool ContainsEnemy(YaotideSim sim, int id)
{
    for (int i = 0; i &lt; sim.Enemies.Count; i++)
        if (sim.Enemies[i].Id == id) return true;
```

```
        return false;
    }
    EnemyVisual CreateEnemyVisual(YaotideSim.EnemyInst e)
    {
        int id = e.Id;
        var go = new GameObject("Enemy_" + e.Id, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.sizeDelta = e.Boss ? new Vector2(160, 160) : (e.Elite ? new Vector2(110, 110) : new Vector2(70, 70));
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = e.Boss ? PalGold : (e.Elite ? PalGreen : new Color(0.35f, 0.32f, 0.42f));
        if (e.Boss)
            MakeText(go.transform, " ", 64, Vector2.zero, TextAnchor.MiddleCenter, 160, 160).color = PalBlack;
        else if (e.Elite)
            MakeText(go.transform, " ", 44, Vector2.zero, TextAnchor.MiddleCenter, 110, 110).color = PalBlack;
        var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
        hpGo.transform.SetParent(go.transform, false);
        var hpRt = (RectTransform)hpGo.transform;
        hpRt.anchorMin = new Vector2(0f, 1f);
        hpRt.anchorMax = new Vector2(1f, 1f);
        hpRt.anchoredPosition = new Vector2(0, -6);
        hpRt.sizeDelta = new Vector2(-8, 10);
        var fill = hpGo.GetComponent&lt;Image&gt;();
        fill.sprite = _square;
        fill.type = Image.Type.Filled;
        fill.fillMethod = Image.FillMethod.Horizontal;
        fill.color = new Color(0.2f, 0.9f, 0.3f);
        var focusLabel = MakeText(go.transform, "  ", 34, new Vector2(0, 120), TextAnchor.MiddleCenter, 220, 50);
        focusLabel.color = PalGold;
        focusLabel.gameObject.SetActive(false);
        go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; EnemyTapped?.Invoke(id));
        return new EnemyVisual
        {
            Go = go,
            Rt = rt,
            Fill = fill,
            FocusLabel = focusLabel.gameObject,
            X = e.X,
            Y = e.Y,
            Hp = e.Hp,
            MaxHp = e.MaxHp,
        };
    }
    // ——— HUD     ———
    public void SetWave(int w) =&gt; _waveText.text = "   " + w + "/" + _totalWaves;
    public void SetCalamity(string name) =&gt; _calamityText.text = name;
    public void SetLives(int l) =&gt; _livesText.text = "   " + l;
    public void SetCharms(int c) =&gt; _charmText.text = "  " + c;
```

```
public void SetKills(int k) =&gt; _killsText.text = "   " + k;
public void SetHint(string hint) =&gt; _hintText.text = hint;
public void HideIntro() =&gt; _introOverlay.gameObject.SetActive(false);
public void SetTower(int slot, bool isAoE, int yaoId)
{
    _slotImages[slot].color = isAoE ? PalRed : PalBlue;
    var def = YaoCodex.ById(yaoId);
    string label = def != null &amp;&amp; def.Name.Length &gt; 0 ? def.Name.Substring(0, 1) : (isAoE ? " " : " "
    _slotLabels[slot].text = label;
    _slotLabels[slot].color = PalBlack;
    _slotLabels[slot].fontSize = 64;
}
public void PlayExecute()
{
    StartCoroutine(ExecuteRoutine());
}
IEnumerator ExecuteRoutine()
{
    _executeOverlay.gameObject.SetActive(true);
    yield return new WaitForSeconds(0.5f);
    _executeOverlay.gameObject.SetActive(false);
}
public void ShowSettle(YaotideRunResult result)
{
    _settleTitle.text = result.Won ? "    " : "    ";
    string seal = result.SealedYaoId &gt; 0 &amp;&amp; YaoCodex.ById(result.SealedYaoId) != null
        ? YaoCodex.ById(result.SealedYaoId).Name + "     "
        : "";
    _settleStats.text = result.Won
        ? $"   {result.WaveReached}/{_totalWaves} ·    {result.Kills} ·   +{result.CharmsEarned}\n&lt;size=40&gt
        : $"   {result.WaveReached}/{_totalWaves} ·    {result.Kills} ·   +{result.CharmsEarned}\n&lt;size=40&gt
    _settlePanel.gameObject.SetActive(true);
}
// ————    ————
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = color;
    return rt;
}
void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
{
    var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
```

```
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(420, 120);
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = _square;
            image.color = PalGreen;
            MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120);
            go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
        }
        static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
        {
            var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(w, h);
            var t = go.GetComponent&lt;Text&gt;();
            t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            t.text = text;
            t.fontSize = size;
            t.alignment = anchor;
            t.color = new Color(0.91f, 0.89f, 0.85f);
            t.supportRichText = true;
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
        sealed class EnemyVisual
        {
            public GameObject Go;
            public RectTransform Rt;
            public Image Fill;
            public GameObject FocusLabel;
            public bool LastFocused;
            public bool Focused;
            public float X, Y;
            public float Hp, MaxHp;
        }
    }
}
```